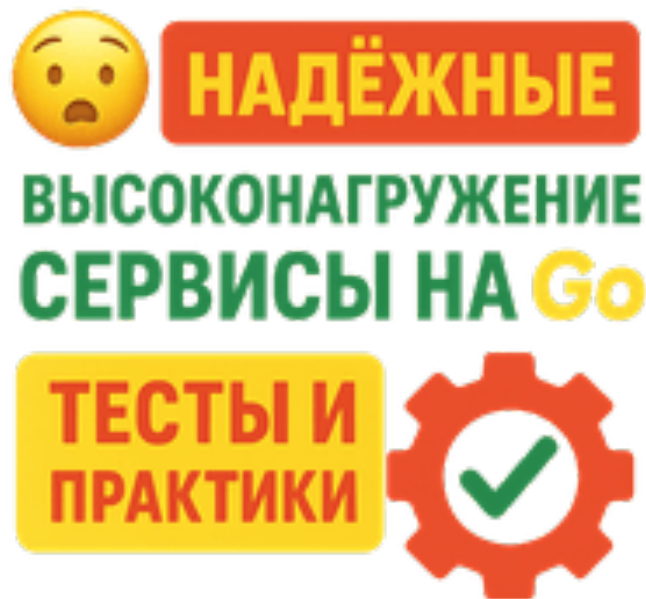




## Разработчик высоконагруженных сервисов на Go

### Модуль 1 - Модуль 1. Введение в высоконагруженные системы

#### Урок 1 - Что такое Highload

##### Шаг 02:

###### Вопрос 1

Что из перечисленного является ключевым признаком highload-системы?

Подсказка: Подумайте, что делает систему высоконагруженной не по объему данных, а по нагрузке на обработку.

**Ответ:**

- **Обслуживание сотен тысяч пользователей одновременно**
- Хранение большого объема данных
- Использование редких технологий
- Наличие монолитной архитектуры

##### Шаг 03:

###### Вопрос 2

Сопоставьте метрики highload-систем с их определениями.

Метрики:

**Ответ:**

- RPS
- Latency
- Throughput
- Availability

Определения:

1. Количество запросов, которые система обрабатывает в секунду.
2. Время ответа сервиса на запрос.
3. Пропускная способность, например в MB/s.
4. Доступность сервиса в процентах, например SLA 99.9%.

 **Подсказка:**

Обратите внимание на то, какая метрика измеряет скорость, а какая стабильность работы.

- **RPS** — Количество запросов, которые система обрабатывает в секунду.
- **Latency** — Время ответа сервиса на запрос.
- **Throughput** — Пропускная способность, например в MB/s.
- **Availability** — Доступность сервиса в процентах, например SLA 99.9%.

 **Шаг 04:**

**Вопрос 3**

Какая из перечисленных систем является примером highload-сервиса?

Подсказка: Подумайте о сервисах, которые обслуживают миллионы пользователей или обрабатывают миллионы запросов в секунду.

**Ответ:**

- **Google Search**
- Небольшой блог на WordPress
- Персональный сайт портфолио
- Локальная база данных на ноутбуке

 **Шаг 05:**

Вопрос 4 : Расположите шаги горизонтального масштабирования highload-сервиса в правильном порядке:

1. Добавление новых серверов
2. Настройка балансировщика нагрузки
3. Перенос состояния или кэша
4. Проверка стабильности работы

 **Подсказка:**

Сначала добавляем ресурсы, затем распределяем нагрузку, потом переносим состояние и проверяем работу.

**Ответ:**

- Добавление новых серверов
- Настройка балансировщика нагрузки
- Перенос состояния или кэша
- Проверка стабильности работы

## 🔗 Шаг 06:

### Вопрос 5

Что из перечисленного помогает снизить нагрузку на highload-сервис?

Подсказка: Подумайте о способах хранения часто запрашиваемых данных ближе к пользователю.

**Ответ:**

- **Использование Redis для кэширования**
- Увеличение размера логов
- Выключение мониторинга
- Добавление ненужных комментариев в код

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте инструменты с их назначением для highload-сервиса:

Инструменты:

**Ответ:**

- Nginx
- Kafka
- Prometheus
- Redis

Назначение:

1. Балансировка нагрузки и обратный прокси
2. Асинхронная обработка сообщений
3. Мониторинг метрик
4. Кэширование данных в памяти

### 🔗 Подсказка:

Подумайте, какой инструмент используется для распределения нагрузки, а какой — для хранения быстрого доступа к данным.

- **Nginx** — Балансировка нагрузки и обратный прокси
- **Kafka** — Асинхронная обработка сообщений
- **Prometheus** — Мониторинг метрик
- **Redis** — Кэширование данных в памяти

## 🔗 Шаг 08:

### Вопрос 7

Как повысить отказоустойчивость highload-сервиса?

Подсказка: Подумайте о методах, которые позволяют системе продолжать работать при отказе одного из компонентов.

**Ответ:**

- **Репликация данных и failover**
- Полное отключение кэширования
- Удаление всех логов
- Снижение числа серверов до одного

## 🔗 Шаг 09:

Вопрос 8 : Расположите этапы обработки запроса highload-сервисом в правильном порядке:

1. Прием запроса сервером
2. Кэширование ответа при необходимости
3. Обработка логики и обращение к базе данных
4. Возврат ответа клиенту

### 🔗 Подсказка:

Сначала сервер получает запрос, затем проверяет кэш, потом выполняет логику, и только после этого отправляет результат.

#### Ответ:

- Прием запроса сервером
- Кэширование ответа при необходимости
- Обработка логики и обращение к базе данных
- Возврат ответа клиенту

## 🔗 Шаг 10:

### Вопрос 9

Сопоставьте метрики latency с их определениями:

Метрики:

#### Ответ:

- P50
- P95
- P99

Определения:

1. **Медианное время ответа** — половина запросов быстрее этого значения
2. 95% запросов быстрее этого времени
3. 99% запросов быстрее этого времени

### 🔗 Подсказка:

Обратите внимание на percentiles и их значение для анализа производительности.

- **P50** — Медианное время ответа — половина запросов быстрее этого значения
- **P95** — 95% запросов быстрее этого времени
- **P99** — 99% запросов быстрее этого времени

## 🔗 Шаг 11:

### Вопрос 10

При проведении финального нагрузочного тестирования Go-сервиса, что следует делать первым?

Подсказка: Подумайте о подготовке окружения и мониторинге перед подачей нагрузки.

#### Ответ:

- **Настроить мониторинг и тестовое окружение, похожее на production**
- Сразу подавать максимальную нагрузку на сервис
- Удалять логи перед тестом
- Отключать кэширование перед тестом

## Урок 2 - Особенности Go в Highload

### Шаг 02:

#### Вопрос 1

Что из перечисленного является ключевым преимуществом Go для highload-систем?

Подсказка: Подумайте о том, как язык позволяет запускать десятки тысяч задач одновременно.

**Ответ:**

- **Лёгкие goroutines и эффективная конкурентность**
- Большие и тяжёлые потоки
- Сложная система управления памятью
- Отсутствие стандартной библиотеки

### Шаг 03:

#### Вопрос 2

Сопоставьте элементы конкурентности Go с их назначением:

Элементы:

**Ответ:**

- goroutine
- channel
- context.Context

Назначение:

1. Лёгкий поток для параллельной работы
2. Обмен данными между goroutines
3. Управление временем жизни и отменой задач

#### Подсказка:

Обратите внимание, что goroutine — это «легкий поток», канал — средство коммуникации, context управляет временем жизни.

- **goroutine** — Лёгкий поток для параллельной работы
- **channel** — Обмен данными между goroutines
- **context.Context** — Управление временем жизни и отменой задач

### Шаг 04:

#### Вопрос 3

Почему в Go можно запускать тысячи goroutines на одном ядре процессора?

Подсказка: Подумайте о размере памяти, необходимой для одной goroutine.

**Ответ:**

- **Они занимают всего ~2 KB памяти каждая**
- Они используют весь стек процессора
- **Каждая goroutine** — отдельный процесс ОС
- Нужен большой heap для запуска одной goroutine

## 🔗 Шаг 05:

Вопрос 4 : Расположите шаги запуска и завершения goroutines в правильном порядке:

1. Создание WaitGroup
2. Добавление числа задач в WaitGroup
3. Запуск goroutine с defer wg.Done()
4. Ожидание завершения всех goroutines через wg.Wait()

### 🔗 Подсказка:

Сначала создаём механизм ожидания, добавляем задачи, запускаем goroutines, затем ждём завершения.

#### Ответ:

- Создание WaitGroup
- Добавление числа задач в WaitGroup
- Запуск goroutine с defer wg.Done()
- Ожидание завершения всех goroutines через wg.Wait()

## 🔗 Шаг 06:

### Вопрос 5

Что из перечисленного является правильным использованием channel в Go?

Подсказка: Подумайте о безопасной передаче данных между goroutines без гонок.

#### Ответ:

- **Передача результата работы одной goroutine другой через channel**
- Использование channel как обычного массива
- Изменение глобальных переменных без синхронизации
- Запуск channel без приёма данных

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте языки с их преимуществами для highload:

Языки:

#### Ответ:

- Go
- Java
- Node.js

Преимущества:

1. Простота, лёгкие goroutines, низкая latency
2. Зрелая экосистема, многопоточность
3. Прост в IO-bound задачах, удобен для async

### 🔗 Подсказка:

Смотрите на особенности конкурентности и области применения.

- **Go** — Простота, лёгкие goroutines, низкая latency
- **Java** — Зрелая экосистема, многопоточность
- **Node.js** — Прост в IO-bound задачах, удобен для async

## 🔗 Шаг 08:

### Вопрос 7

Какой инструмент встроен в Go для анализа производительности highload-систем?

Подсказка: Этот инструмент позволяет измерять CPU, память и блокировки goroutines.

**Ответ:**

- **pprof**
- jvisualvm
- valgrind
- perf

## 🔗 Шаг 09:

Вопрос 8 : Расположите шаги для тестирования 10 000 goroutines с HTTP-запросами:

1. Создание HTTP-запроса в goroutine
2. Настройка WaitGroup
3. Запуск всех goroutines
4. Ожидание завершения всех goroutines

🔗 **Подсказка:**

Сначала готовим механизм синхронизации, потом создаём запросы, запускаем все goroutines, затем ждём завершения.

**Ответ:**

- Настройка WaitGroup
- Создание HTTP-запроса в goroutine
- Запуск всех goroutines
- Ожидание завершения всех goroutines

## 🔗 Шаг 10:

### Вопрос 9

Сопоставьте метрики latency с их определениями:

Метрики:

**Ответ:**

- P50
- P95
- P99

Определения:

1. **Медианное время ответа** — половина запросов быстрее этого значения
2. 95% запросов быстрее этого времени
3. 99% запросов быстрее этого времени

🔗 **Подсказка:**

Обратите внимание на percentiles и их значение для анализа производительности.

- **P50** — Медианное время ответа — половина запросов быстрее этого значения
- **P95** — 95% запросов быстрее этого времени
- **P99** — 99% запросов быстрее этого времени

## 🔊 Шаг 11:

### Вопрос 10

При проведении финального нагрузочного тестирования Go-сервиса, что следует делать первым?

Подсказка: Подумайте о подготовке окружения и мониторинге перед подачей нагрузки.

**Ответ:**

- **Настроить мониторинг и тестовое окружение, похожее на production**
- Сразу подавать максимальную нагрузку на сервис
- Удалять логи перед тестом
- Отключать кэширование перед тестом

## 📖 Урок 3 - Архитектурные принципы Highload

## 🔊 Шаг 02:

### Вопрос 1

Что из перечисленного относится к принципам масштабирования highload-систем?

Подсказка: Подумайте о том, как система увеличивает свои ресурсы для обработки нагрузки.

**Ответ:**

- **Вертикальное и горизонтальное масштабирование**
- Единственный сервер без репликации
- Игнорирование нагрузки на базу данных
- Монолит без модулей и кэша

## 🔊 Шаг 03:

### Вопрос 2

Сопоставьте элементы CAP-теоремы с их определением:

Элементы:

**Ответ:**

- Consistency
- Availability
- Partition tolerance

Определения:

1. Каждый запрос получает ответ, даже при сбое
2. Все узлы видят одни и те же данные
3. Система работает при разделении сети

## 🔊 Подсказка:

Подумайте, что выбирают при проектировании распределённых систем: согласованность, доступность или устойчивость к разделению.

- **Consistency** — Все узлы видят одни и те же данные
- **Availability** — Каждый запрос получает ответ, даже при сбое
- **Partition tolerance** — Система работает при разделении сети

## 🔗 Шаг 04:

### Вопрос 3

Какой подход помогает сделать highload-систему менее блокирующей?

Подсказка: Подумайте о способах обработки задач без ожидания завершения других операций.

**Ответ:**

- **Асинхронная обработка через очереди и события**
- Последовательное выполнение всех операций
- Игнорирование ошибок сети
- Использование единственного потока для всех задач

## 🔗 Шаг 05:

Вопрос 4 : Расположите этапы работы микросервисной системы в правильном порядке:

1. Клиент отправляет запрос через API Gateway
2. Сервис авторизации проверяет пользователя
3. Сервис обработки данных выполняет задачу
4. Результат возвращается клиенту

🔗 **Подсказка:**

Сначала точка входа, затем проверка авторизации, потом обработка данных, и только после этого ответ клиенту.

**Ответ:**

- Клиент отправляет запрос через API Gateway
- Сервис авторизации проверяет пользователя
- Сервис обработки данных выполняет задачу
- Результат возвращается клиенту

## 🔗 Шаг 06:

### Вопрос 5

Какой приём уменьшает количество обращений к базе данных и ускоряет выдачу данных пользователю?

Подсказка: Подумайте о временном хранении часто используемых данных.

**Ответ:**

- **Кэширование (например, Redis)**
- Повторные запросы к SQL при каждом обращении
- Синхронная блокировка всех запросов
- Игнорирование данных с ошибками

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте тип системы с её характеристиками по CAP-теореме:

Типы:

**Ответ:**

- CP-система
- AP-система

Характеристики:

1. Жертвуем доступностью ради согласованности
2. Жертвуем строгой согласованностью ради доступности

🔗 **Подсказка:**

Обратите внимание, что разные базы выбирают разные компромиссы.

- **СР-система** — Жертвуем доступностью ради согласованности
- **АР-система** — Жертвуем строгой согласованностью ради доступности

🔗 **Шаг 08:**

### Вопрос 7

Что характеризует архитектуру event-driven в highload-системах?

Подсказка: Подумайте, как система реагирует на события и уведомляет другие компоненты.

**Ответ:**

- **Компоненты реагируют на события через очереди или подписки**
- Все операции выполняются последовательно в одном сервисе
- Игнорирование сетевых сообщений
- Использование только SQL без очередей

🔗 **Шаг 09:**

Вопрос 8 : Расположите шаги обработки запроса через Feed Service с кэшем Redis:

1. Проверка кэша Redis
2. Если данных нет, запрос к базе данных
3. Запись полученных данных в кэш
4. Отправка ответа клиенту

🔗 **Подсказка:**

Сначала проверяем кэш, если нет — обращаемся к БД, потом обновляем кэш и возвращаем ответ.

**Ответ:**

- Проверка кэша Redis
- Если данных нет, запрос к базе данных
- Запись полученных данных в кэш
- Отправка ответа клиенту

🔗 **Шаг 10:**

### Вопрос 9

Какой приём помогает системе оставаться доступной при сбое отдельных компонентов?

Подсказка: Подумайте о резервных копиях и fallback-логике.

**Ответ:**

- **Репликация, fallback, graceful degradation**
- Игнорирование сбоев и остановка системы
- Синхронная блокировка всех сервисов
- Использование одного сервера без резервов

## 🔗 Шаг 11:

### Вопрос 10

Сопоставьте тип данных с рекомендуемым хранилищем для highload-системы:

Типы данных:

**Ответ:**

- Транзакционные данные
- Часто читаемые данные (кэш)
- События и логи

Хранилища:

1. PostgreSQL
2. Redis
3. Kafka

### 🔗 Подсказка:

Смотрите на характер данных: критичность транзакций, скорость чтения и поток событий.

- **Транзакционные данные** — PostgreSQL
- **Часто читаемые данные (кэш)** — Redis
- **События и логи** — Kafka

## 📖 Урок 4 - Обзор реальных highload-систем

## 🔗 Шаг 02:

### Вопрос 1

Какой подход применяет Google для обеспечения высокой производительности поиска при сотнях тысяч запросов в секунду?

Подсказка: Подумайте о распределении нагрузки и масштабировании по географии.

**Ответ:**

- **Распределённые дата-центры и собственные сети**
- Монолитный сервер без репликации
- Локальные файлы на одном сервере
- Обработка только одного запроса в секунду

## 🔗 Шаг 03:

### Вопрос 2

Какая технология помогает Netflix доставлять видео миллионам пользователей без задержек?

Подсказка: Подумайте о CDN и распределении контента.

**Ответ:**

- **CDN OpenConnect и микросервисы**
- Единый сервер в одном дата-центре
- Отправка видео по email
- Сохранение видео только локально у пользователя

## 🔗 Шаг 04:

### Вопрос 3

Сопоставьте сервисы ВКонтакте с их назначением:

Сервисы:

**Ответ:**

- Auth Service
- Messages Service
- Feed Service
- Media Service

Назначение:

1. Обработка фото и видео
2. Публикация и выдача новостей в ленте с кэшем Redis
3. Регистрация и вход пользователей
4. Отправка и получение сообщений через очередь Kafka

 **Подсказка:**

Подумайте, какой сервис отвечает за авторизацию, какой за медиа, а какой за сообщения.

- **Auth Service** — Регистрация и вход пользователей
- **Messages Service** — Отправка и получение сообщений через очередь Kafka
- **Feed Service** — Публикация и выдача новостей в ленте с кэшем Redis
- **Media Service** — Обработка фото и видео

 **Шаг 05:**

#### Вопрос 4

Что помогает Ozon обрабатывать миллионы заказов за короткое время во время распродаж?

Подсказка: Подумайте о микросервисах, очередях и оркестрации.

**Ответ:**

- **Микросервисы, очереди Kafka и Kubernetes**
- Один сервер с монолитным приложением
- Локальная база без репликации
- Отправка заказов вручную

 **Шаг 06:**

Вопрос 5 : Расположите шаги обработки пользовательского запроса в упрощённой архитектуре VK:

1. Клиент отправляет запрос через API Gateway
2. Auth Service проверяет пользователя
3. Feed Service выдаёт ленту с кэшем Redis
4. Media Service обрабатывает фото/видео при необходимости
5. Ответ возвращается клиенту

 **Подсказка:**

Сначала точка входа, потом проверка авторизации, затем обработка ленты и медиа, потом возвращение ответа.

**Ответ:**

- Клиент отправляет запрос через API Gateway
- Auth Service проверяет пользователя

- Feed Service выдаёт ленту с кэшем Redis
- Media Service обрабатывает фото/видео при необходимости
- Ответ возвращается клиенту

## Шаг 07:

### Вопрос 6

Какое узкое место чаще всего возникает в highload-системах при пиковых нагрузках?

Подсказка: Подумайте, что может стать бутылочным горлышком при миллионах запросов в секунду.

**Ответ:**

- **База данных и очередь сообщений**
- Клиентские приложения
- Документы в локальной папке
- Мониторинг Prometheus

## Шаг 08:

### Вопрос 7

Сопоставьте highload-проекты с архитектурными решениями:

Проекты:

**Ответ:**

- Google Search
- Netflix
- ВКонтакте
- Ozon

Решения:

1. CDN, микросервисы, autoscaling
2. Распределённые дата-центры, MapReduce, BigTable
3. Горизонтальное масштабирование, Tarantool, Redis
4. Микросервисы, Kafka, Kubernetes

### Подсказка:

Обратите внимание на характер нагрузки: поиск, видео, социальная сеть или e-commerce.

- **Google Search** — Распределённые дата-центры, MapReduce, BigTable
- **Netflix** — CDN, микросервисы, autoscaling
- **ВКонтакте** — Горизонтальное масштабирование, Tarantool, Redis
- **Ozon** — Микросервисы, Kafka, Kubernetes

## Шаг 09:

### Вопрос 8

Для чего в Messages Service ВКонтакте используют Tarantool и Redis?

Подсказка: Подумайте, как ускорить отправку и получение сообщений миллионами пользователей.

**Ответ:**

- **Для кэширования и быстрой обработки очередей сообщений**
- Для хранения видео и фото

- Для отправки email пользователям
- Для анализа логов Prometheus

#### 🔗 Шаг 10:

Вопрос 9 : Расположите шаги при обработке запроса пользователя в Netflix:

1. Клиент запрашивает видео
2. Проверка кэша CDN
3. Если видео нет в кэше, загрузка с основного сервера
4. Передача видео клиенту

#### 🔗 Подсказка:

Сначала клиент запрашивает видео, потом проверяем CDN, при необходимости обращаемся к серверу и только после этого отдаём видео пользователю.

**Ответ:**

- Клиент запрашивает видео
- Проверка кэша CDN
- Если видео нет в кэше, загрузка с основного сервера
- Передача видео клиенту

#### 🔗 Шаг 11:

##### Вопрос 10

Какой общий вывод можно сделать из обзора архитектур реальных highload-систем?

Подсказка: Подумайте о стандартах и повторяющихся решениях.

**Ответ:**

- **Микросервисы + очереди + кэш + мониторинг встречаются почти везде**
- Все системы используют монолит на одном сервере
- Нет необходимости в кэшировании и очередях
- Облачные решения не применяются

### 📖 Урок 5 - Метрики и SLA

#### 🔗 Шаг 02:

##### Вопрос 1

Что измеряет метрика RPS в высоконагруженной системе?

Подсказка: Подумайте, сколько действий система обрабатывает за единицу времени.

**Ответ:**

- **Количество запросов в секунду**
- Время отклика сервера
- Объём переданных данных за минуту
- Процент успешных запросов

#### 🔗 Шаг 03:

##### Вопрос 2

Что показывает метрика Latency?

Подсказка: Подумайте о том, сколько времени проходит между запросом и ответом.

**Ответ:**

- **Время отклика системы на запрос**
- Количество пользователей онлайн
- Объём данных на диске
- Число ошибок в месяц

🔗 **Шаг 04:**

### **Вопрос 3**

Что измеряет Throughput?

Подсказка: Подумайте о том, сколько данных проходит через систему за единицу времени.

**Ответ:**

- **Объём данных или количество запросов в единицу времени**
- Время отклика на один запрос
- Процент ошибок
- Загрузка процессора

🔗 **Шаг 05:**

### **Вопрос 4**

Что измеряет Error Rate в системе?

Подсказка: Подумайте, какой процент запросов не выполняется успешно.

**Ответ:**

- **Процент неуспешных запросов**
- Количество пользователей на сайте
- Скорость передачи данных
- Время отклика P50

🔗 **Шаг 06:**

### **Вопрос 5**

Сопоставьте понятия SLA, SLO и SLI с их определениями:

Понятия:

**Ответ:**

- SLA
- SLO
- SLI

Определения:

1. Формальное соглашение с клиентом о доступности сервиса
2. Целевой уровень сервиса внутри компании
3. Конкретная метрика для измерения показателей сервиса

🔗 **Подсказка:**

Обратите внимание на различие: SLA — внешнее соглашение, SLO — цель для команды, SLI — конкретный индикатор.

- **SLA** — Формальное соглашение с клиентом о доступности сервиса
- **SLO** — Целевой уровень сервиса внутри компании

- **SLI** — Конкретная метрика для измерения показателей сервиса

#### 🔗 Шаг 07:

##### Вопрос 6

Почему важно смотреть percentiles (P50, P95, P99) вместо среднего значения latency?

Подсказка: Подумайте о пиковых нагрузках и скрытых задержках.

**Ответ:**

- **Чтобы увидеть пиковые задержки, которые среднее скрывает**
- Чтобы получить меньше информации о системе
- Чтобы уменьшить RPS
- Чтобы измерять только ошибки

#### 🔗 Шаг 08:

Вопрос 7 : Расположите шаги измерения RPS и latency для Go HTTP-сервиса в правильном порядке:

1. Запуск сервера Go
2. Запуск нагрузочного теста (например, wrk)
3. Сбор метрик Requests/sec и latency
4. Анализ percentiles и вычисление SLA/SLO

🔗 **Подсказка:**

Сначала сервис должен работать, потом тестируем, затем собираем метрики и анализируем результаты.

**Ответ:**

- Запуск сервера Go
- Запуск нагрузочного теста (например, wrk)
- Сбор метрик Requests/sec и latency
- Анализ percentiles и вычисление SLA/SLO

#### 🔗 Шаг 09:

##### Вопрос 8

Если SLO сервиса — 95% запросов должны выполняться <50ms, а P95 latency = 45ms, выполнено ли SLO?

Подсказка: P95 показывает, сколько запросов укладываются в предел.

**Ответ:**

- **Да, SLO выполнено**
- Нет, SLO не выполнено
- Невозможно сказать без RPS
- SLO относится только к error rate

#### 🔗 Шаг 10:

##### Вопрос 9

Если SLA сервиса = 99.9% доступности, сколько времени в месяц сервис может быть недоступен?

Подсказка: Месяц ≈ 30 дней, вычисляем 0.1% от общего времени.

**Ответ:**

- **Максимум 43 минуты**
- Максимум 4 минуты
- Максимум 1 час
- Сервис не может быть недоступен

 **Шаг 11:**

**Вопрос 10**

Что важно учитывать при установке SLA/SLO в highload-системах?

Подсказка: Подумайте о компромиссах между временем отклика, доступностью и нагрузкой.

**Ответ:**

- **Trade-offs между latency, availability и throughput**
- Только загрузку CPU
- Только количество пользователей
- Только error rate

## **Модуль 2 - Архитектура и проектирование систем**

### **Урок 1 - Декомпозиция сервисов и API-дизайн**

 **Шаг 02:**

**Вопрос 1**

Какова основная цель декомпозиции монолитной системы на сервисы?

Подсказка: Подумайте, зачем разделять систему на отдельные части с понятной ответственностью.

**Ответ:**

- **Масштабируемость и независимый деплой отдельных частей**
- Усложнение архитектуры
- Объединение всех функций в один сервис
- Увеличение времени отклика

 **Шаг 03:**

**Вопрос 2**

Какое преимущество REST API при проектировании высоконагруженных сервисов?

Подсказка: Подумайте о простоте интеграции и совместимости с разными клиентами.

**Ответ:**

- **Простота и широкая поддержка клиентов**
- Бинарный протокол и строгие типы
- Сложность дебага без инструментов
- Неэффективность при малой нагрузке

 **Шаг 04:**

**Вопрос 3**

Какое преимущество gRPC перед REST в highload-системах?

Подсказка: Подумайте о производительности и типизации данных.

**Ответ:**

- **Быстрый бинарный протокол с строгой типизацией**
- Проще тестировать с curl
- Улучшает работу с JSON
- Не требует версионирования API

🔗 **Шаг 05:**

#### **Вопрос 4**

В чем основное преимущество GraphQL для клиентов сервисов?

Подсказка: Подумайте о том, как клиент выбирает нужные данные.

**Ответ:**

- **Клиент сам выбирает, какие поля получать**
- Сервис автоматически кэширует все запросы
- Используется только JSON
- Подходит только для малых систем

🔗 **Шаг 06:**

#### **Вопрос 5**

Сопоставьте тип API с его характеристикой:

**Ответ:**

- REST
  - gRPC
  - GraphQL
1. Простой HTTP/JSON интерфейс с широкой поддержкой
  2. Быстрый бинарный протокол с строго типизированными сообщениями
  3. Позволяет клиенту запрашивать только необходимые поля

🔗 **Подсказка:**

REST — популярный HTTP, gRPC — быстрый бинарный, GraphQL — выбор полей клиентом.

- **REST** — Простой HTTP/JSON интерфейс с широкой поддержкой
- **gRPC** — Быстрый бинарный протокол с строго типизированными сообщениями
- **GraphQL** — Позволяет клиенту запрашивать только необходимые поля

🔗 **Шаг 07:**

#### **Вопрос 6**

Почему важно версионировать API сервисов?

Подсказка: Подумайте о совместимости старых клиентов при изменении функционала.

**Ответ:**

- **Чтобы изменения не ломали старых клиентов**
- Чтобы увеличить нагрузку на сервер
- Чтобы уменьшить количество эндпоинтов
- Чтобы использовать только JSON

## 🔗 Шаг 08:

### Вопрос 7

Что означает идемпотентность в контексте API?

Подсказка: Подумайте о повторном вызове одного и того же запроса.

**Ответ:**

- **Повторный запрос не изменяет результат**
- Каждый запрос увеличивает нагрузку вдвое
- Запрос нельзя повторять
- Результат запроса всегда уникален

## 🔗 Шаг 09:

### Вопрос 8

Что произойдет, если отправить POST /posts с корректным JSON телом?

Подсказка: Подумайте о создании нового объекта и возврате ответа.

**Ответ:**

- **Создается новый пост и возвращается с ID**
- Возвращается список всех пользователей
- Сервер завершает работу
- Происходит ошибка компиляции Go

## 🔗 Шаг 10:

Вопрос 9 : Расположите шаги по созданию Users Service на Go в правильном порядке:

1. Определение структуры User и хранилища
2. Реализация GET /users и POST /users
3. Запуск HTTP-сервера
4. Тестирование с curl или Postman

🔗 **Подсказка:**

Сначала структура, потом реализация эндпоинтов, запуск и проверка.

**Ответ:**

- Определение структуры User и хранилища
- Реализация GET /users и POST /users
- Запуск HTTP-сервера
- Тестирование с curl или Postman

## 🔗 Шаг 11:

### Вопрос 10

Почему важно разделять Users Service и Posts Service на отдельные сервисы?

Подсказка: Подумайте о масштабируемости и независимом развитии каждого функционала.

**Ответ:**

- **Чтобы можно было масштабировать и деплоить независимо**
- Чтобы уменьшить количество пользователей
- Чтобы все запросы шли через один эндпоинт
- Чтобы объединить все данные в одну таблицу

## Урок 2 - Балансировка нагрузки и кэширование

### Шаг 02:

#### Вопрос 1

Какова основная цель балансировки нагрузки в высоконагруженных системах?

Подсказка: Подумайте о распределении запросов между серверами и повышении отказоустойчивости.

**Ответ:**

- **Равномерное распределение запросов и повышение доступности**
- Увеличение нагрузки на один сервер
- Сокращение количества серверов
- Игнорирование отказов отдельных серверов

### Шаг 03:

#### Вопрос 2

В чем отличие L4 и L7 балансировки нагрузки?

Подсказка: L4 работает на уровне TCP/UDP, L7 — на уровне HTTP.

**Ответ:**

- **L4** — транспортный уровень, L7 — прикладной HTTP уровень
- **L4** — HTTP, L7 — TCP/UDP
- L4 всегда медленнее L7
- L7 не может балансировать по URL

### Шаг 04:

#### Вопрос 3

Какой алгоритм балансировки отдаёт запрос серверу с наименьшим количеством активных соединений?

Подсказка: Подумайте о динамическом распределении нагрузки в реальном времени.

**Ответ:**

- **Least Connections**
- Round Robin
- IP Hash
- Random

### Шаг 05:

#### Вопрос 4

Зачем использовать кэширование в highload-сервисах?

Подсказка: Подумайте о снижении нагрузки на базу и ускорении ответа сервиса.

**Ответ:**

- **Снизить нагрузку на базу и ускорить ответы**
- Сделать сервис медленнее
- Увеличить количество запросов к базе
- Избежать балансировки нагрузки

## 🔗 Шаг 06:

### Вопрос 5

В чем разница между локальным кэшем приложения и внешним кэшем вроде Redis?

Подсказка: Локальный — в памяти приложения, внешний — общий для всех экземпляров сервиса.

**Ответ:**

- **Локальный кэш хранится в памяти одного приложения, внешний доступен всем сервисам**
- Локальный кэш всегда быстрее Redis
- Внешний кэш не может хранить TTL
- Локальный кэш используется только для POST-запросов

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте уровень кэширования с примером:

**Ответ:**

- Локальный кэш
  - Внешний кэш
  - CDN/edge-кэш
1. map + mutex в Go
  2. Redis или Memcached
  3. Cloudflare, NGINX caching

🔗 **Подсказка:**

Подумайте о месте хранения данных и доступе к ним для разных сервисов.

- **Локальный кэш** — map + mutex в Go
- **Внешний кэш** — Redis или Memcached
- **CDN/edge-кэш** — Cloudflare, NGINX caching

## 🔗 Шаг 08:

### Вопрос 7

Что делает стратегия TTL в кэшировании?

Подсказка: Подумайте о сроке жизни данных в кэше.

**Ответ:**

- **Данные автоматически удаляются после заданного времени**
- Всегда сохраняются навсегда
- Удаляются при каждом GET-запросе
- Сохраняются только в памяти клиента

## 🔗 Шаг 09:

### Вопрос 8

Что произойдет с RPS и latency, если добавить кэширование для часто запрашиваемых данных?

Подсказка: Подумайте о снижении нагрузки на базу и ускорении отклика.

**Ответ:**

- **RPS увеличится, latency уменьшится**

- RPS уменьшится, latency увеличится
- Никаких изменений не будет
- Сервис перестанет работать

#### 🔧 Шаг 10:

Вопрос 9 : Расположите шаги по реализации простого Round-Robin load balancer на Go:

1. Создать список адресов серверов
2. Инициализировать счётчик запросов
3. Определять `target = servers[counter % len(servers)]` и увеличивать counter
4. Перенаправлять HTTP-запросы на target

#### 💡 Подсказка:

Сначала список серверов, потом счётчик, вычисление target и перенаправление запроса.

#### Ответ:

- Создать список адресов серверов
- Инициализировать счётчик запросов
- Определять `target = servers[counter % len(servers)]` и увеличивать counter
- Перенаправлять HTTP-запросы на target

#### 🔧 Шаг 11:

#### Вопрос 10

Как кэширование и балансировка нагрузки влияют на производительность и отказоустойчивость сервиса?

Подсказка: Подумайте о снижении нагрузки на базу, ускорении отклика и равномерном распределении запросов.

#### Ответ:

- **Увеличивают RPS, уменьшают latency, повышают отказоустойчивость**
- Замедляют сервис и повышают нагрузку на базу
- Не оказывают никакого влияния
- Используются только для логирования

### 📖 Урок 3 - Дизайн API под нагрузкой

#### 🔧 Шаг 02:

#### Вопрос 1

Какова основная цель высоконагруженного API?

Подсказка: Подумайте о стабильной работе сервиса под большим количеством запросов.

#### Ответ:

- **Обеспечить масштабируемость и отказоустойчивость**
- Сделать API сложным для использования
- Игнорировать нагрузку и ошибки
- Позволять любому клиенту менять данные без ограничений

#### 🔧 Шаг 03:

#### Вопрос 2

Что означает идемпотентность в API?

Подсказка: Подумайте о повторном запросе, который не меняет результат.

**Ответ:**

- **Повторный запрос не изменяет результат**
- Каждый запрос создаёт новый результат
- API возвращает случайные данные
- Запросы блокируются после первого вызова

🔗 **Шаг 04:**

### Вопрос 3

Зачем нужно версионирование API?

Подсказка: Подумайте о развитии сервиса без поломки клиентов.

**Ответ:**

- **Чтобы развивать сервис без нарушения работы клиентов**
- Чтобы усложнить работу разработчиков
- Чтобы делать API медленнее
- Чтобы полностью блокировать старые клиенты

🔗 **Шаг 05:**

### Вопрос 4

Что делает rate limiting в высоконагруженном API?

Подсказка: Подумайте о защите сервиса от перегрузки.

**Ответ:**

- **Ограничивает количество запросов на пользователя/IP**
- Увеличивает количество запросов
- Меняет структуру данных
- Всегда возвращает 500 ошибку

🔗 **Шаг 06:**

### Вопрос 5

Зачем применять пагинацию и limit/offset в API?

Подсказка: Подумайте о больших списках данных и оптимизации трафика.

**Ответ:**

- **Чтобы уменьшить payload и контролировать размер возвращаемых данных**
- Чтобы всегда возвращать все данные сразу
- Чтобы блокировать пользователей
- Чтобы усложнить запросы

🔗 **Шаг 07:**

### Вопрос 6

Сопоставьте подход к API с его особенностью:

**Ответ:**

- REST
- gRPC

- GraphQL

1. HTTP/JSON, простое для внешних клиентов
2. Бинарный протокол, высокая производительность, поддержка потоков
3. Клиент выбирает данные, может быть тяжёлым при highload

#### 🔗 Подсказка:

Подумайте о формате данных и сценариях использования.

- **REST** — HTTP/JSON, простое для внешних клиентов
- **gRPC** — Бинарный протокол, высокая производительность, поддержка потоков
- **GraphQL** — Клиент выбирает данные, может быть тяжёлым при highload

#### 🔗 Шаг 08:

##### Вопрос 7

Какой HTTP-код уместно возвращать при превышении rate limit?

Подсказка: Подумайте о сигнале клиенту уменьшить нагрузку.

**Ответ:**

- **429 Too Many Requests**
- 400 Bad Request
- 500 Internal Server Error
- 200 OK

#### 🔗 Шаг 09:

##### Вопрос 8

Что даёт объединение нескольких операций в один batch-запрос?

Подсказка: Подумайте о снижении количества сетевых вызовов.

**Ответ:**

- **Уменьшает количество сетевых вызовов и нагрузку на API**
- Увеличивает нагрузку на клиента
- Всегда замедляет API
- Позволяет игнорировать rate limit

#### 🔗 Шаг 10:

Вопрос 9 : Расположите шаги по внедрению rate limiting на Go:

1. Создать лимитер с ограничением req/sec
2. Проверять, разрешён ли запрос limiter.Allow()
3. Если превышение, вернуть 429 Too Many Requests
4. Если разрешено, обработать запрос как обычно

#### 🔗 Подсказка:

Сначала создаём лимитер, потом проверяем, и в зависимости от результата возвращаем ошибку или продолжаем обработку.

**Ответ:**

- Создать лимитер с ограничением req/sec
- Проверять, разрешён ли запрос limiter.Allow()
- Если превышение, вернуть 429 Too Many Requests

- Если разрешено, обработать запрос как обычно

## Шаг 11:

### Вопрос 10

Как версионирование, rate limiting и batch-запросы влияют на API под нагрузкой?

Подсказка: Подумайте о надежности, стабильности и эффективности.

**Ответ:**

- **Повышают стабильность, уменьшают нагрузку и сохраняют совместимость клиентов**
- Замедляют работу API и создают ошибки
- Не оказывают влияния
- Используются только для логирования

## Урок 4 - Очереди и event-driven архитектура

## Шаг 02:

### Вопрос 1

Что является ключевой идеей event-driven архитектуры?

Подсказка: Подумайте о том, как сервисы взаимодействуют между собой.

**Ответ:**

- **Сервисы взаимодействуют через события, а не прямые запросы**
- Все сервисы обмениваются данными напрямую через HTTP
- Сервисы работают только синхронно
- События игнорируются при высокой нагрузке

## Шаг 03:

### Вопрос 2

Что делает Producer в event-driven системе?

Подсказка: Подумайте о том, кто создаёт события.

**Ответ:**

- **Публикует события в очередь или брокер**
- Подписывается на события и обрабатывает их
- Удаляет старые события из брокера
- Обрабатывает события синхронно без очереди

## Шаг 04:

### Вопрос 3

Что делает Consumer в event-driven системе?

Подсказка: Подумайте о том, кто реагирует на события.

**Ответ:**

- **Подписывается на события и выполняет обработку**
- Создаёт новые события для очереди
- Удаляет старые события без обработки
- Игнорирует очередь при высокой нагрузке

## 🔗 Шаг 05:

### Вопрос 4

Что обеспечивает Message Broker в event-driven системе?

Подсказка: Подумайте о доставке и очередности событий.

**Ответ:**

- **Гарантирует доставку и очередность событий между сервисами**
- Всегда обрабатывает события быстрее всех сервисов
- Игнорирует ошибки при доставке
- Удаляет все события после публикации

## 🔗 Шаг 06:

### Вопрос 5

Какое преимущество даёт event-driven архитектура?

Подсказка: Подумайте о масштабируемости и асинхронности.

**Ответ:**

- **Высокая масштабируемость и асинхронная обработка**
- Все запросы обрабатываются синхронно
- Уменьшение отказоустойчивости
- Сложность взаимодействия сервисов увеличивается без пользы

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте тип очереди с его особенностью:

**Ответ:**

- Persistent
  - In-memory
  - Priority
1. Сохраняет события на диске, гарантирует доставку (Kafka)
  2. Быстрая, но данные теряются при падении сервиса (NATS)
  3. Обрабатывает задачи в порядке приоритета

🔗 **Подсказка:**

Подумайте о надежности, скорости и порядке обработки.

- **Persistent** — Сохраняет события на диске, гарантирует доставку (Kafka)
- **In-memory** — Быстрая, но данные теряются при падении сервиса (NATS)
- **Priority** — Обрабатывает задачи в порядке приоритета

## 🔗 Шаг 08:

### Вопрос 7

Какой брокер сообщений подходит для high-throughput и долговременного хранения событий?

Подсказка: Подумайте о Kafka, RabbitMQ и NATS.

**Ответ:**

- **Kafka**
- NATS

- RabbitMQ (в базовой конфигурации)
- HTTP REST сервис без очереди

#### Шаг 09:

##### Вопрос 8

Почему асинхронная обработка событий важна в высоконагруженных системах?

Подсказка: Подумайте о независимости обработки от клиента.

**Ответ:**

- **Позволяет обрабатывать события независимо от клиента и разгружает сервисы**
- Замедляет обработку запросов клиента
- Увеличивает синхронные блокировки
- Всегда уменьшает производительность очереди

#### Шаг 10:

Вопрос 9 : Расположите шаги работы сервиса с очередью:

1. Producer публикует событие
2. Message Broker сохраняет/передаёт событие
3. Consumer получает событие
4. Consumer обрабатывает событие

 **Подсказка:**

Сначала событие создаётся, затем передаётся брокером, потом потребляется и обрабатывается.

**Ответ:**

- Producer публикует событие
- Message Broker сохраняет/передаёт событие
- Consumer получает событие
- Consumer обрабатывает событие

#### Шаг 11:

##### Вопрос 10

Как использование очередей сообщений влияет на высоконагруженные системы?

Подсказка: Подумайте о производительности, отказоустойчивости и масштабируемости.

**Ответ:**

- **Сглаживает нагрузку, повышает отказоустойчивость и масштабируемость**
- Всегда снижает производительность
- Упрощает синхронное взаимодействие
- Не влияет на производительность

### Урок 5 - Кэширование в многослойной архитектуре

#### Шаг 02:

##### Вопрос 1

Какова основная цель многослойного кэширования?

Подсказка: Подумайте о том, как ускорить ответы и снизить нагрузку на базу.

**Ответ:**

- **Снизить нагрузку на базу и ускорить ответы клиентов**
- Увеличить количество запросов к базе
- Игнорировать базу данных и работать только с кэшем
- Всегда хранить данные только в локальной памяти

🔗 **Шаг 03:**

### **Вопрос 2**

Какой уровень кэша общий для всех экземпляров сервиса?

Подсказка: Подумайте о Redis или Memcached.

**Ответ:**

- **Распределённый кэш (Redis, Memcached)**
- Локальный кэш приложения
- CDN / edge-кэш
- Только база данных

🔗 **Шаг 04:**

### **Вопрос 3**

Какая особенность локального кэша приложения?

Подсказка: Подумайте о скорости и доступности между сервисами.

**Ответ:**

- **Быстрый доступ, но не делится между экземплярами сервисов**
- Общий для всех сервисов и всегда консистентный
- Используется только для статики и файлов
- Позволяет сохранять данные на диске постоянно

🔗 **Шаг 05:**

### **Вопрос 4**

Для чего используется CDN / edge-кэш?

Подсказка: Подумайте о публичном контенте и задержках сети.

**Ответ:**

- **Разгрузка backend и снижение latency для клиентов**
- Увеличение нагрузки на базу данных
- Хранение только внутренних данных сервиса
- Замедление ответа сервиса при каждом запросе

🔗 **Шаг 06:**

### **Вопрос 5**

Как работает стратегия Cache-aside (Lazy loading)?

Подсказка: Подумайте о том, когда данные попадают в кэш.

**Ответ:**

- **Данные запрашиваются из базы при первом обращении и сохраняются в кэше**
- Данные записываются сразу только в кэш без базы
- Кэш обновляется только вручную администратором

- Все данные всегда читаются только из кэша

#### 🔗 Шаг 07:

##### Вопрос 6

Что происходит при использовании стратегии Write-through?

Подсказка: Подумайте о синхронизации кэша и базы.

**Ответ:**

- **Кэш обновляется одновременно с записью в базу**
- Кэш обновляется только после запроса клиента
- Кэш обновляется асинхронно, база сразу нет
- Данные никогда не сохраняются в кэше

#### 🔗 Шаг 08:

##### Вопрос 7

Как работает стратегия Write-back / Write-behind?

Подсказка: Подумайте о порядке записи в кэш и базу.

**Ответ:**

- **Кэш обновляется первым, запись в базу происходит асинхронно**
- База обновляется первой, кэш игнорируется
- Данные не попадают в кэш
- Кэш обновляется вручную после каждого запроса

#### 🔗 Шаг 09:

##### Вопрос 8

Сопоставьте слой кэша с его характеристикой:

**Ответ:**

- Локальный кэш
  - Redis / Memcached
  - CDN
1. Быстрый доступ в рамках одного сервиса
  2. Общий для всех сервисов, распределённый
  3. Снижает задержку и разгружает backend для публичного контента

🔗 **Подсказка:**

Подумайте о скорости, консистентности и области применения.

- **Локальный кэш** — Быстрый доступ в рамках одного сервиса
- **Redis / Memcached** — Общий для всех сервисов, распределённый
- **CDN** — Снижает задержку и разгружает backend для публичного контента

#### 🔗 Шаг 10:

Вопрос 9 : Расположите слои обработки запроса с кэшем в правильном порядке:

1. Проверка локального кэша
2. Проверка Redis кэша
3. Запрос к базе данных

#### 4. Обновление Redis и локального кэша

##### **Подсказка:**

Сначала ищем данные ближе всего к сервису, потом дальше, и при отсутствии обновляем все слои.

##### **Ответ:**

- Проверка локального кэша
- Проверка Redis кэша
- Запрос к базе данных
- Обновление Redis и локального кэша

##### **Шаг 11:**

##### **Вопрос 10**

Какой trade-off существует при выборе времени жизни (TTL) кэша?

Подсказка: Подумайте о балансе между нагрузкой на базу и актуальностью данных.

##### **Ответ:**

- **Большой TTL → меньше нагрузки на базу, но данные могут быть устаревшими**
- Малый TTL → всегда актуальные данные без нагрузки на базу
- TTL не влияет на консистентность
- TTL только увеличивает скорость кэша без минусов

## **Модуль 3 - Конкурентность в Go**

### **Урок 1 - Горутины, каналы, context**

##### **Шаг 02:**

##### **Вопрос 1**

Что такое горутина в Go?

Подсказка: Подумайте о лёгких потоках, которые планировщик Go распределяет по ядрам.

##### **Ответ:**

- **Лёгкий поток, выполняемый планировщиком Go**
- Тяжёлый поток ОС, который требует отдельного ядра
- Синхронная функция, выполняющаяся последовательно
- Специальный тип канала для передачи данных

##### **Шаг 03:**

##### **Вопрос 2**

Какую роль играют каналы (chan) в Go?

Подсказка: Подумайте о безопасной передаче данных между горутинами.

##### **Ответ:**

- **Безопасная передача данных между горутинами**
- Замедление работы программы
- Хранение данных в базе
- Создание потоков ОС

## 🔗 Шаг 04:

### Вопрос 3

Что даёт использование буферизированного канала?

Подсказка: Подумайте, как отправка и получение данных могут происходить без немедленной блокировки.

**Ответ:**

- **Позволяет отправлять несколько элементов без мгновенного получения**
- Ускоряет выполнение без ограничений
- Гарантирует, что данные всегда записываются в файл
- Блокирует все горютины до получения результата

## 🔗 Шаг 05:

### Вопрос 4

Для чего используется пакет context в Go?

Подсказка: Подумайте о времени жизни задач и возможности их отмены.

**Ответ:**

- **Управление временем жизни горютин и отмена операций**
- Передача данных между процессами ОС
- Синхронная обработка HTTP-запросов без отмен
- Ускорение работы каналов

## 🔗 Шаг 06:

### Вопрос 5

Что произойдёт, если context отменится во время работы горютины?

Подсказка: Подумайте о select и канале ctx.Done().

**Ответ:**

- **Горютина получает сигнал и может корректно завершиться**
- Горютина продолжает работать, игнорируя context
- Программа аварийно завершится
- Данные теряются без уведомления

## 🔗 Шаг 07:

Вопрос 6 : Расположите действия при обработке задач горютинами с context в правильном порядке:

1. Горютина получает задачу из канала jobs
2. Горютина проверяет ctx.Done() для возможной отмены
3. Горютина обрабатывает задачу и отправляет результат в канал results
4. Горютина завершается корректно

🔗 **Подсказка:**

Сначала получаем задачу, проверяем отмену, затем обрабатываем и отправляем результат.

**Ответ:**

- Горютина получает задачу из канала jobs
- Горютина проверяет ctx.Done() для возможной отмены

- Горутина обрабатывает задачу и отправляет результат в канал results
- Горутина завершается корректно

#### 🔗 Шаг 08:

##### Вопрос 7

Сопоставьте термин с его описанием:

**Ответ:**

- Горутина
  - Канал (chan)
  - Буферизированный канал
  - Context
1. Лёгкий поток Go
  2. Механизм передачи данных между горутинами
  3. Канал с возможностью временного хранения элементов
  4. Управление временем жизни и отменой задач

#### 🔗 Подсказка:

Подумайте о функции каждого инструмента для конкурентного выполнения задач.

- **Горутина** — Лёгкий поток Go
- **Канал (chan)** — Механизм передачи данных между горутинами
- **Буферизированный канал** — Канал с возможностью временного хранения элементов
- **Context** — Управление временем жизни и отменой задач

#### 🔗 Шаг 09:

##### Вопрос 8

Что произойдет, если горутина не успевает выполнить задачу до истечения таймута context?

Подсказка: Подумайте о select и ctx.Done().

**Ответ:**

- **Горутина будет отменена, операция прервется**
- Горутина выполнится несмотря на таймаут
- Программа зависнет навсегда
- Результат будет отправлен позже автоматически

#### 🔗 Шаг 10:

##### Вопрос 9

Почему стоит контролировать количество одновременно запущенных горутин?

Подсказка: Подумайте о ресурсах памяти и производительности.

**Ответ:**

- **Слишком много горутин может перегрузить планировщик и память**
- Количество горутин не влияет на работу программы
- Меньше горутин всегда хуже производительность
- Горутин автоматически ограничиваются количеством ядер

#### 🔗 Шаг 11:

##### Вопрос 10

Какая главная польза от использования горутин, каналов и context вместе?

Подсказка: Подумайте о параллельной и безопасной обработке задач.

**Ответ:**

- **Обработка множества задач параллельно и безопасно с контролем времени жизни**
- Ускорение работы программы без ограничения ресурсов
- Избежание использования каналов и mutex
- Запуск задач только последовательно

## Урок 2 - Синхронизация и атомарные операции

### Шаг 02:

#### Вопрос 1

Что может произойти, если несколько горутин одновременно изменяют одну переменную без синхронизации?

Подсказка: Подумайте о том, что результат может зависеть от порядка выполнения горутин.

**Ответ:**

- **Гонки данных (race condition)**
- Синхронизация автоматически произойдёт
- Программа завершится аварийно
- Значение переменной всегда будет корректным

### Шаг 03:

#### Вопрос 2

Для чего используется sync.Mutex в Go?

Подсказка: Подумайте о блокировке ресурса для одной горутин.

**Ответ:**

- **Чтобы заблокировать доступ к ресурсу одной горутине**
- Чтобы ускорить выполнение программы
- Чтобы создать горутину
- Чтобы передавать данные между горутинами

### Шаг 04:

#### Вопрос 3

Что делает sync.RWMutex в отличие от обычного Mutex?

Подсказка: Подумайте о возможности одновременного чтения несколькими горутинами.

**Ответ:**

- **Позволяет нескольким горутинам читать одновременно, запись блокирует всех**
- Запрещает чтение и запись одновременно без ограничений
- Автоматически повышает производительность
- Используется только для атомарных операций

### Шаг 05:

#### Вопрос 4

Для чего используется sync.WaitGroup в Go?

Подсказка: Подумайте о том, как дождаться завершения группы горутин.

**Ответ:**

- Чтобы дождаться завершения всех горутин в группе
- Чтобы синхронизировать доступ к переменной
- Чтобы создавать новые горутин
- Чтобы выполнять атомарные операции

🔗 **Шаг 06:**

### Вопрос 5

Что делают атомарные операции из пакета sync/atomic?

Подсказка: Подумайте о безопасном изменении значения без Mutex.

**Ответ:**

- Позволяют безопасно изменять значения без Mutex
- Создают горутин автоматически
- Блокируют все операции чтения
- Гарантируют, что все горутин завершатся синхронно

🔗 **Шаг 07:**

Вопрос 6: Расположите действия при безопасном увеличении счетчика с Mutex:

1. Захват блокировки (mu.Lock())
2. Увеличение значения счетчика
3. Освобождение блокировки (mu.Unlock())
4. Продолжение работы программы

🔗 **Подсказка:**

Сначала блокируем ресурс, затем изменяем значение, освобождаем и продолжаем работу.

**Ответ:**

- Захват блокировки (mu.Lock())
- Увеличение значения счетчика
- Освобождение блокировки (mu.Unlock())
- Продолжение работы программы

🔗 **Шаг 08:**

### Вопрос 7

Сопоставьте инструмент с его назначением:

**Ответ:**

- Mutex
  - RWMutex
  - WaitGroup
  - sync/atomic
1. Блокировка доступа к ресурсу одной горутине
  2. Чтение несколькими горутинами, запись блокирует всех
  3. Ожидание завершения группы горутин
  4. Атомарные операции без Mutex

🔗 **Подсказка:**

Подумайте, какой инструмент решает какую задачу синхронизации.

- **Mutex** — Блокировка доступа к ресурсу одной горутине
- **RWMutex** — Чтение несколькими горутинami, запись блокирует всех
- **WaitGroup** — Ожидание завершения группы горутин
- **sync/atomic** — Атомарные операции без Mutex

#### 🔗 Шаг 09:

##### Вопрос 8

Когда лучше использовать атомарные операции вместо Mutex?

Подсказка: Подумайте о простых числовых операциях, где важна скорость.

**Ответ:**

- **Для простых операций, где важна производительность**
- Для сложной логики с несколькими переменными
- Для блокировки чтения и записи одновременно
- Для ожидания завершения горутин

#### 🔗 Шаг 10:

##### Вопрос 9

Что покажет программа с 100 горутинami, каждая увеличивает общий счетчик без синхронизации, при запуске с флагом -race?

Подсказка: Подумайте о гонках данных и несогласованном доступе.

**Ответ:**

- **Флаг -race обнаружит гонки данных**
- Счетчик всегда будет равен 100000
- Программа аварийно завершится
- Mutex автоматически исправит гонки

#### 🔗 Шаг 11:

##### Вопрос 10

Какая главная цель использования синхронизации и атомарных операций в Go?

Подсказка: Подумайте о безопасной работе с общими данными в многопоточной среде.

**Ответ:**

- **Обеспечить безопасное и корректное изменение общих данных**
- Сделать программу однопоточной
- Ускорить выполнение без ограничений
- Автоматически уменьшить использование памяти

### 📖 Урок 3 - Advanced concurrency patterns

#### 🔗 Шаг 02:

##### Вопрос 1

Что делает паттерн Worker Pool в Go?

Подсказка: Подумайте о распределении задач между ограниченным числом горутин.

**Ответ:**

- **Распределяет задачи между фиксированным числом горутин**
- Создаёт новые горутины для каждой задачи без ограничения
- Объединяет результаты нескольких функций
- Служит только для синхронизации данных

### 🔮 Шаг 03:

#### Вопрос 2

Что такое Fan-out и Fan-in в Go?

Подсказка: Подумайте о параллельной обработке данных и сборе результатов.

**Ответ:**

- **Fan-out: несколько горутин обрабатывают задачи, Fan-in: собирает результаты в один канал**
- Fan-out: объединяет каналы, Fan-in: создаёт новые горутины
- Fan-out: увеличивает скорость горутин, Fan-in: замедляет их
- **Fan-out и Fan-in** — это библиотеки Go

### 🔮 Шаг 04:

Вопрос 3: Расположите этапы работы pipeline правильно:

1. Получение данных из входного канала
2. Обработка данных
3. Передача данных в следующий канал
4. Вывод результатов

🔮 **Подсказка:**

Pipeline обрабатывает данные по цепочке: вход → обработка → следующий этап → выход.

**Ответ:**

- Получение данных из входного канала
- Обработка данных
- Передача данных в следующий канал
- Вывод результатов

### 🔮 Шаг 05:

#### Вопрос 4

Зачем используется rate limiting в Go?

Подсказка: Подумайте о том, чтобы не перегружать ресурсы при высокой нагрузке.

**Ответ:**

- **Ограничить скорость выполнения задач**
- Ускорить все горутины одновременно
- Объединять результаты нескольких воркеров
- Создавать новые каналы автоматически

### 🔮 Шаг 06:

#### Вопрос 5

Для чего в Go используется конструкция select с каналами?

Подсказка: Подумайте о работе с несколькими каналами одновременно и таймаутах.

**Ответ:**

- **Обработка нескольких каналов одновременно с таймаутами и отменой**
- Создание новых горутин по очереди
- Ограничение скорости выполнения задач
- Сбор всех результатов в массив

#### **Шаг 07:**

Вопрос 6: Расположите этапы работы worker pool с fan-in правильно:

1. Создание канала jobs
2. Запуск воркеров
3. Отправка задач в канал jobs
4. Сбор результатов из канала results

 **Подсказка:**

Сначала создаем канал, запускаем воркеров, отправляем задачи, потом собираем результаты (fan-in).

**Ответ:**

- Создание канала jobs
- Запуск воркеров
- Отправка задач в канал jobs
- Сбор результатов из канала results

#### **Шаг 08:**

##### **Вопрос 7**

Сопоставьте паттерн с его назначением:

**Ответ:**

- Worker Pool
  - Fan-out/Fan-in
  - Pipeline
  - Rate Limiting
  - Select Patterns
1. Ограничение числа одновременно выполняемых задач
  2. Параллельная обработка задач и объединение результатов
  3. Цепочка обработки данных через несколько этапов
  4. Ограничение скорости выполнения задач
  5. Обработка нескольких каналов с таймаутами и отменой

 **Подсказка:**

Подумайте о функциональности каждого паттерна.

- **Worker Pool** — Ограничение числа одновременно выполняемых задач
- **Fan-out/Fan-in** — Параллельная обработка задач и объединение результатов
- **Pipeline** — Цепочка обработки данных через несколько этапов
- **Rate Limiting** — Ограничение скорости выполнения задач
- **Select Patterns** — Обработка нескольких каналов с таймаутами и отменой

## Шаг 09:

### Вопрос 8

Почему использование fan-out ускоряет обработку задач?

Подсказка: Подумайте о нескольких горутинах, работающих параллельно.

**Ответ:**

- **Потому что несколько горутин обрабатывают задачи одновременно**
- Потому что уменьшается количество задач
- Потому что каналы становятся ненужными
- Потому что увеличивается задержка передачи данных

## Шаг 10:

### Вопрос 9

Зачем использовать context.WithTimeout при обработке задач в Go?

Подсказка: Подумайте о предотвращении зависания горутин.

**Ответ:**

- **Чтобы отменять задачи, если они выполняются слишком долго**
- Чтобы ускорить выполнение всех горутин
- Чтобы автоматически увеличивать число воркеров
- Чтобы объединять каналы результатов

## Шаг 11:

### Вопрос 10

Какова главная цель использования продвинутых паттернов конкурентности в Go?

Подсказка: Подумайте о производительности и безопасности обработки параллельных задач.

**Ответ:**

- **Эффективная, безопасная и контролируемая обработка большого числа параллельных задач**
- Сделать программу однопоточной
- Автоматически уменьшить использование памяти
- Обходиться без каналов и горутин

## Урок 4 - Проектирование высоконагруженных worker-систем

## Шаг 02:

### Вопрос 1

Что является основной ролью worker в высоконагруженной системе?

Подсказка: Подумайте, кто обрабатывает задачи и возвращает результаты.

**Ответ:**

- **Обрабатывает задачи и отправляет результаты**
- Генерирует задачи и помещает их в очередь
- Собирает результаты и публикует события
- Балансирует нагрузку между очередями

### Шаг 03:

#### Вопрос 2

Какова роль очереди (Queue / Broker) в worker-системе?

Подсказка: Подумайте о распределении задач между воркерами.

**Ответ:**

- **Распределяет задачи между воркерами и управляет очередью**
- Обработывает задачи напрямую
- Генерирует новые воркеры автоматически
- Отменяет задачи при превышении времени

### Шаг 04:

Вопрос 3: Расположите этапы работы высоконагруженной worker-системы в правильном порядке:

1. Producer создаёт задачи
2. Задачи помещаются в очередь
3. Worker обрабатывает задачи
4. Aggregator собирает результаты

 **Подсказка:**

Сначала задачи создаются, потом помещаются в очередь, далее обрабатываются воркерами, и только потом результаты собираются.

**Ответ:**

- Producer создаёт задачи
- Задачи помещаются в очередь
- Worker обрабатывает задачи
- Aggregator собирает результаты

### Шаг 05:

#### Вопрос 4

Что подразумевается под горизонтальной масштабируемостью worker-системы?

Подсказка: Подумайте о добавлении воркеров без изменения кода.

**Ответ:**

- **Возможность добавлять воркеров без изменения кода**
- Увеличение памяти каждого воркера
- Создание отдельной очереди для каждой задачи
- Использование однопоточного исполнения

### Шаг 06:

#### Вопрос 5

Почему важна идемпотентность задач при повторной обработке?

Подсказка: Подумайте о том, чтобы повторная попытка не ломала данные.

**Ответ:**

- **Чтобы повторная обработка задачи не меняла результат некорректно**
- Чтобы задача выполнялась быстрее

- Чтобы можно было удалить задачу из очереди
- Чтобы воркеры не использовали каналы

#### 🔗 Шаг 07:

Вопрос 6: Расположите правильный порядок обработки задачи с очередью и retry:

1. Задача помещается в очередь
2. Worker берёт задачу из очереди
3. Обработка задачи
4. **При ошибке** — задача помещается обратно (retry)
5. **При успехе** — отправка результата в aggregator

#### 🔗 Подсказка:

Очередь управляет задачами, воркер пытается обработать, при ошибке возвращаем, при успехе собираем результат.

#### Ответ:

- Задача помещается в очередь
- Worker берёт задачу из очереди
- Обработка задачи
- **При ошибке** — задача помещается обратно (retry)
- **При успехе** — отправка результата в aggregator

#### 🔗 Шаг 08:

#### Вопрос 7

Сопоставьте компонент worker-системы с его функцией:

#### Ответ:

- Producer
  - Queue/Broker
  - Worker
  - Aggregator
1. Генерация задач
  2. Распределение задач и управление очередью
  3. Обработка задач
  4. Сбор и публикация результатов

#### 🔗 Подсказка:

Подумайте о потоках данных: создание → очередь → обработка → сбор.

- **Producer** — Генерация задач
- **Queue/Broker** — Распределение задач и управление очередью
- **Worker** — Обработка задач
- **Aggregator** — Сбор и публикация результатов

#### 🔗 Шаг 09:

#### Вопрос 8

Что такое backpressure в контексте worker-систем?

Подсказка: Подумайте о защите системы от перегрузки.

**Ответ:**

- **Механизм ограничения нагрузки, чтобы предотвратить перегрузку очереди и воркеров**
- Автоматическое ускорение обработки задач
- Удаление задач из очереди при перегрузке
- Создание новых воркеров без ограничения

 **Шаг 10:**

#### **Вопрос 9**

Почему важно использовать context и таймауты для каждой задачи?

Подсказка: Подумайте о контроле времени выполнения и отмене зависших задач.

**Ответ:**

- **Чтобы отменять задачи, которые выполняются слишком долго, и контролировать время обработки**
- Чтобы ускорить выполнение всех воркеров одновременно
- Чтобы объединять все каналы в один
- Чтобы создать больше задач автоматически

 **Шаг 11:**

#### **Вопрос 10**

Какова главная цель проектирования высоконагруженных worker-систем на Go?

Подсказка: Подумайте о масштабируемости, отказоустойчивости и контроле обработки задач.

**Ответ:**

- **Обеспечить безопасную, масштабируемую и контролируемую обработку большого числа задач параллельно**
- Сделать систему однопоточной для простоты
- Автоматически уменьшить задержки до нуля
- Обходиться без каналов и context

## **Модуль 4 - Работа с данными**

### **Урок 1 - SQL и NoSQL под нагрузкой**

 **Шаг 02:**

#### **Вопрос 1**

Какое преимущество SQL баз данных (PostgreSQL, MySQL) особенно полезно при сложных транзакциях?

Подсказка: Подумайте о согласованности и целостности данных.

**Ответ:**

- **ACID-транзакции, обеспечивающие согласованность**
- Гибкая схема хранения данных
- Очень высокая скорость записи без индексов
- Поддержка eventual consistency

## 🔮 Шаг 03:

### Вопрос 2

Какая сильная сторона Redis как NoSQL базы в highload-системах?

Подсказка: Подумайте о скорости операций и кешировании.

**Ответ:**

- **Очень высокая скорость чтения и записи для key-value данных**
- Сложные JOIN и агрегаты
- Согласованность ACID по умолчанию
- Горизонтальное шардирование невозможно

## 🔮 Шаг 04:

Вопрос 3: Расположите шаги обращения к данным с использованием кеширования Redis:

1. Проверка наличия данных в Redis
2. **Если данных нет** — обращение к PostgreSQL
3. Сохранение данных в Redis с TTL
4. Возврат данных клиенту

🔮 **Подсказка:**

Сначала проверяем кэш, если нет — идем в базу, затем кешируем и возвращаем клиенту.

**Ответ:**

- Проверка наличия данных в Redis
- **Если данных нет** — обращение к PostgreSQL
- Сохранение данных в Redis с TTL
- Возврат данных клиенту

## 🔮 Шаг 05:

### Вопрос 4

Зачем использовать индексы в SQL базах под нагрузкой?

Подсказка: Подумайте о быстром поиске и сокращении времени выполнения запросов.

**Ответ:**

- **Для ускорения поиска и фильтрации данных**
- Чтобы уменьшить размер базы данных
- Для обеспечения eventual consistency
- Для хранения кэша в памяти

## 🔮 Шаг 06:

### Вопрос 5

Зачем используют репликацию (master-slave) в SQL под highload?

Подсказка: Подумайте о распределении чтений и повышении доступности.

**Ответ:**

- **Чтобы распределять чтения между репликами и снижать нагрузку на основной сервер**
- Чтобы ускорить запись транзакций на master
- Чтобы отключить индексы на slave

- Чтобы кэшировать данные в памяти

### Шаг 07:

Вопрос 6: Расположите шаги выполнения batch-запроса к базе данных:

1. Сбор нескольких ID для запроса
2. Отправка одного SQL-запроса с множеством ID
3. Получение всех результатов сразу
4. Обработка результатов в приложении

### Подсказка:

Сначала собираем данные, делаем один запрос, затем получаем результаты и обрабатываем их.

### Ответ:

- Сбор нескольких ID для запроса
- Отправка одного SQL-запроса с множеством ID
- Получение всех результатов сразу
- Обработка результатов в приложении

## Урок 2 - Кэш и очереди

### Шаг 02:

#### Вопрос 1

Для чего используют кэш в высоконагруженных Go-сервисах?

Подсказка: Подумайте о сокращении времени доступа к данным и уменьшении нагрузки на базу.

### Ответ:

- **Для быстрого доступа к часто используемым данным**
- Чтобы полностью заменить базу данных
- Для хранения только статических файлов
- Для обработки задач в очередях

### Шаг 03:

#### Вопрос 2

Какая особенность распределенного кэша (Redis, Memcached) по сравнению с локальным кэшем в Go?

Подсказка: Подумайте о доступе из нескольких экземпляров приложения.

### Ответ:

- **Доступен для всех сервисов, разделяемый между экземплярами**
- Быстрее локального кэша
- Не поддерживает TTL
- Хранится только на диске

### Шаг 04:

Вопрос 3: Расположите стратегии кэширования по порядку их применения:

1. Cache-aside (ленивое)
2. Write-through

- 3. Write-back
- 4. TTL/LRU/LFU управление временем жизни

 **Подсказка:**

Подумайте о том, как данные сначала загружаются, затем обновляются, и как контролируется их срок жизни.

**Ответ:**

- Cache-aside (ленивое)
- Write-through
- Write-back
- TTL/LRU/LFU управление временем жизни

 **Шаг 05:**

**Вопрос 4**

Почему используют очереди в высоконагруженных Go-сервисах?

Подсказка: Подумайте о разгрузке сервисов и асинхронной обработке.

**Ответ:**

- **Для асинхронной обработки задач и снижения нагрузки на сервисы**
- Для синхронного блокирующего вызова функций
- Чтобы кэшировать статические файлы
- Чтобы хранить данные пользователей на диске

 **Шаг 06:**

**Вопрос 5**

Зачем используют dead-letter очередь и retry в системах с очередями?

Подсказка: Подумайте о надежности и обработке ошибок.

**Ответ:**

- **Чтобы повторно обработать задачи при ошибках и сохранять неудачные задачи для анализа**
- Чтобы ускорить чтение кэша
- Чтобы гарантировать моментальное выполнение задачи
- Чтобы полностью исключить использование базы данных

 **Шаг 07:**

Вопрос 6: Расположите шаги обработки заказа через очередь и кэш:

1. Пользователь создает заказ
2. Сервис публикует заказ в очередь
3. Worker обрабатывает заказ
4. Обновление кэша с результатом
5. Клиент получает данные из кэша

 **Подсказка:**

Сначала заказ отправляется в очередь, затем обрабатывается, кэш обновляется, и клиент получает быстрый ответ.

**Ответ:**

- Пользователь создает заказ

- Сервис публикует заказ в очередь
- Worker обрабатывает заказ
- Обновление кэша с результатом
- Клиент получает данные из кэша

## 🔗 Шаг 08:

### Вопрос 7

Сопоставьте тип кэша с его характеристикой:

**Ответ:**

- Локальный кэш
- Распределенный кэш
- CDN/edge-кэш

1. Быстрый доступ внутри одного приложения
2. Доступ для всех сервисов, общий между инстансами
3. Кэш для статики и публичного контента на периферии сети

🔗 **Подсказка:**

Подумайте о скорости доступа и области видимости кэша.

- **Локальный кэш** — Быстрый доступ внутри одного приложения
- **Распределенный кэш** — Доступ для всех сервисов, общий между инстансами
- **CDN/edge-кэш** — Кэш для статики и публичного контента на периферии сети

## 🔗 Шаг 09:

### Вопрос 8

Для чего задают TTL у объектов в Redis?

Подсказка: Подумайте о контроле времени жизни и очистке устаревших данных.

**Ответ:**

- **Чтобы автоматически удалять устаревшие данные и освобождать память**
- Чтобы навсегда хранить данные в кэше
- Чтобы ускорить запись в базу данных
- Чтобы блокировать изменения данных

## 🔗 Шаг 10:

### Вопрос 9

Почему важно использовать backpressure или rate limiting при работе с очередями?

Подсказка: Подумайте о защите сервиса от перегрузки.

**Ответ:**

- **Чтобы не перегружать сервис и обеспечивать стабильную обработку задач**
- Чтобы ускорить запись в кэш
- Чтобы сразу завершить все задачи
- Чтобы игнорировать ошибки воркеров

## 🔗 Шаг 11:

### Вопрос 10

Какой главный trade-off при использовании кэша и очередей?

Подсказка: Подумайте о скорости ответа, консистентности данных и надежности.

**Ответ:**

- **Скорость vs консистентность/надежность: кэш ускоряет чтение, очереди обеспечивают асинхронность, но могут быть задержки или несогласованность**
- Использование кэша всегда снижает надежность
- Очереди гарантируют моментальное выполнение задач без ошибок
- Кэш и очереди одинаково подходят для всех сценариев без компромиссов

## Урок 3 - Шардирование и репликация в практике

### Шаг 02:

#### Вопрос 1

Зачем используют репликацию в highload-системах?

Подсказка: Подумайте о нагрузке на чтение и отказоустойчивости.

**Ответ:**

- **Чтобы распределять нагрузку на чтение и повысить отказоустойчивость**
- Чтобы полностью дублировать записи для ускорения записи
- Чтобы заменить кэширование
- Чтобы объединять несколько баз данных в одну

### Шаг 03:

#### Вопрос 2

Какая разница между синхронной и асинхронной репликацией?

Подсказка: Подумайте о консистентности данных и задержках.

**Ответ:**

- **Синхронная гарантирует консистентность, асинхронная — выше производительность с возможной задержкой**
- Асинхронная гарантирует консистентность, синхронная нет
- Оба типа всегда идентичны
- Синхронная используется только для шардирования

### Шаг 04:

#### Вопрос 3

Что такое шардирование в базах данных?

Подсказка: Подумайте о разделении данных для горизонтального масштабирования.

**Ответ:**

- **Разделение данных на несколько шардов для масштабирования хранения и записи**
- Полная репликация базы на всех серверах
- Использование кэша вместо БД
- Оптимизация индексов в одном сервере

### Шаг 05:

Вопрос 4: Расположите шаги записи и чтения с использованием Primary-Replica:

1. Клиент пишет данные

2. Запись в Primary DB
3. Синхронизация с репликами
4. Чтение данных с реплик

💡 **Подсказка:**

Сначала запись идет в основной сервер, затем данные распространяются на реплики, после чего клиенты могут читать с реплик.

**Ответ:**

- Клиент пишет данные
- Запись в Primary DB
- Синхронизация с репликами
- Чтение данных с реплик

💡 **Шаг 06:**

**Вопрос 5**

Как выбрать правильный шард для записи данных пользователя?

Подсказка: Подумайте о хэшировании или простом остатке по ID.

**Ответ:**

- **Использовать shard key, например `user_id % N`, для выбора шарда**
- Выбирать случайный шард без ключа
- Всегда писать в первый шард
- Не использовать шардирование, писать в Primary напрямую

💡 **Шаг 07:**

**Вопрос 6**

Сопоставьте сущность с её ролью:

**Ответ:**

- Primary DB
  - Replica DB
  - Shard
1. Содержит основную запись данных, все write операции идут сюда
  2. Копия данных для распределения нагрузки на чтение
  3. Часть данных для горизонтального масштабирования

💡 **Подсказка:**

Подумайте о разделении ответственности между записью, чтением и масштабированием.

- **Primary DB** — Содержит основную запись данных, все write операции идут сюда
- **Replica DB** — Копия данных для распределения нагрузки на чтение
- **Shard** — Часть данных для горизонтального масштабирования

💡 **Шаг 08:**

**Вопрос 7**

Что такое fallback при работе с шардированными и реплицированными БД?

Подсказка: Подумайте о доступности данных при сбое.

**Ответ:**

- **Переключение на другой шард или реплику при недоступности**
- Удаление данных при ошибке
- Синхронизация с Primary перед каждой записью
- Отключение всех клиентов до восстановления Primary

🔗 **Шаг 09:**

### Вопрос 8

Какой trade-off возникает при использовании реплик и шардов?

Подсказка: Подумайте о консистентности, задержке и производительности.

**Ответ:**

- **Баланс между консистентностью, задержкой и производительностью: шардирование увеличивает масштаб, реплики ускоряют чтение, но возможны рассинхронизации**
- Использование реплик полностью исключает задержку
- Шарды всегда повышают консистентность
- Trade-off отсутствует при использовании Primary-Replica

🔗 **Шаг 10:**

### Вопрос 9

Зачем выбирать случайную реплику для чтения?

Подсказка: Подумайте о распределении нагрузки между репликами.

**Ответ:**

- **Чтобы равномерно распределять нагрузку на чтение между репликами**
- Чтобы гарантировать консистентность чтения
- Чтобы ускорить запись в Primary
- Чтобы уменьшить количество шардов

🔗 **Шаг 11:**

### Вопрос 10

Как правильно масштабировать запись и чтение в highload Go-системе с шардированием и репликами?

Подсказка: Подумайте о shard key, Primary для записи, реплики для чтения и fallback.

**Ответ:**

- **Использовать shard key для записи, Primary для writes, распределять чтение по репликам, fallback при сбое**
- Писать все данные только в Primary, не использовать реплики
- Использовать случайные шардирования без логики
- Чтение всегда только с одного сервера без балансировки

## 📖 Урок 4 - Event sourcing и CQRS

🔗 **Шаг 02:**

### Вопрос 1

Что является основной идеей Event Sourcing?

Подсказка: Подумайте о том, как хранить историю изменений системы.

**Ответ:**

- **Хранение всех изменений состояния системы как последовательности событий**
- Сохранение только текущего состояния объекта
- Использование кэша для ускорения доступа к данным
- Запись данных напрямую в базу без событий

### Шаг 03:

#### **Вопрос 2**

Какие преимущества дает Event Sourcing?

Подсказка: Подумайте о аудите, реконструкции состояния и интеграции новых представлений.

**Ответ:**

- **Полная история изменений для аудита и отладки**
- **Возможность реконструировать состояние из событий**
- **Легко интегрировать новые projections без изменения старых данных**
- Снижает сложность системы

### Шаг 04:

#### **Вопрос 3**

Что означает CQRS в highload-системах?

Подсказка: Разделение операций чтения и записи.

**Ответ:**

- **Разделение команд (записи) и запросов (чтения) для масштабирования и отказоустойчивости**
- Использование только одной базы для чтения и записи
- Сохранение всех данных в кэше без базы
- Отмена всех изменений при ошибках чтения

### Шаг 05:

#### **Вопрос 4**

Сопоставьте события с действиями в системе:

**Ответ:**

- UserCreated
  - UserEmailUpdated
  - OrderCreated
  - OrderPaid
1. Создание нового пользователя
  2. Обновление email пользователя
  3. Создание нового заказа
  4. Отметка заказа как оплаченного

### Подсказка:

Название события часто отражает действие, которое оно описывает.

- **UserCreated** — Создание нового пользователя
- **UserEmailUpdated** — Обновление email пользователя

- **OrderCreated** — Создание нового заказа
- **OrderPaid** — Отметка заказа как оплаченного

#### 🔗 Шаг 06:

##### Вопрос 5

В каком порядке нужно применять события для восстановления состояния пользователя?

Подсказка: Сначала создаем пользователя, затем применяем обновления.

**Ответ:**

- **Сначала UserCreated, затем UserEmailUpdated**
- Сначала UserEmailUpdated, затем UserCreated
- Применять в случайном порядке
- Только последнее событие

#### 🔗 Шаг 07:

##### Вопрос 6

Какие преимущества дает использование CQRS?

Подсказка: Подумайте о масштабировании и оптимизации нагрузки.

**Ответ:**

- **Масштабирование чтения и записи отдельно**
- **Оптимизация под разные нагрузки**
- Упрощение структуры базы данных
- Полная консистентность без задержек

#### 🔗 Шаг 08:

Вопрос 7: Расположите шаги обработки команды через Event Sourcing и CQRS:

1. Команда отправлена в Command side
2. Command side генерирует событие
3. Событие сохраняется в Event Store
4. Query side обновляет проекции для чтения

🔗 **Подсказка:**

Команды создают события, которые записываются и затем проекции обновляются для быстрого чтения.

**Ответ:**

- Команда отправлена в Command side
- Command side генерирует событие
- Событие сохраняется в Event Store
- Query side обновляет проекции для чтения

#### 🔗 Шаг 09:

##### Вопрос 8

Какой trade-off возникает при использовании Event Sourcing?

Подсказка: Подумайте о производительности и сложности обработки событий.

**Ответ:**

- **Необходимость обработки большого количества событий для реконструкции состояния и усложнение системы**
- Отсутствие преимуществ по сравнению с обычной БД
- Упрощение кода и мгновенная консистентность
- Полное устранение ошибок в системе

🔗 **Шаг 10:**

#### **Вопрос 9**

Зачем строить проекции на Query side?

Подсказка: Подумайте о быстром доступе к данным для чтения.

**Ответ:**

- **Чтобы обеспечить быстрые запросы без пересчета всех событий**
- Чтобы записывать данные напрямую в Event Store
- Чтобы уменьшить число событий в хранилище
- Чтобы полностью исключить eventual consistency

🔗 **Шаг 11:**

#### **Вопрос 10**

Как правильно масштабировать запись и чтение с Event Sourcing и CQRS в Go?

Подсказка: Используйте Event Store для записи событий, Command side для генерации событий, Query side для проекций, учитывайте eventual consistency.

**Ответ:**

- **Command side генерирует события -> сохраняем в Event Store -> Query side строит проекции для чтения, учесть eventual consistency**
- Прямое обновление базы без событий
- Чтение только с Event Store без проекций
- Игнорирование eventual consistency и проекций

## 📁 **Модуль 5 - Масштабирование и отказоустойчивость**

### 📖 **Урок 1 - Паттерны отказоустойчивости**

🔗 **Шаг 02:**

#### **Вопрос 1**

В вашей системе на Go иногда случаются временные ошибки сети. Какой первый и самый простой паттерн отказоустойчивости стоит попробовать применить?

Подсказка: Подумайте о повторных попытках запроса.

**Ответ:**

- **Retry (повторные попытки)**
- Circuit Breaker
- Bulkhead
- Fallback

## 🔧 Шаг 03:

### Вопрос 2

Расставьте шаги реализации Retry с exponential backoff в правильном порядке:

1. Выполнить запрос
2. **Если ошибка** — подождать случайную задержку, увеличивая её экспоненциально
3. Повторить попытку, пока не достигнут лимит
4. **Если успех** — завершить выполнение

Подсказка: backoff нужен, чтобы снизить нагрузку при множественных ошибках.

**Ответ:**

- Выполнить запрос
- **Если ошибка** — подождать случайную задержку, увеличивая её экспоненциально
- Повторить попытку, пока не достигнут лимит
- **Если успех** — завершить выполнение

## 🔧 Шаг 04:

### Вопрос 3

Сопоставьте паттерны с их назначением:

**Ответ:**

- Retry
  - Circuit Breaker
  - Fallback
  - Bulkhead
1. Повторные попытки при временных сбоях
  2. Изоляция компонентов для предотвращения распространения ошибки
  3. Отключение проблемного сервиса после порога ошибок
  4. Альтернативный ответ при недоступности сервиса

Подсказка: Подумайте, как действовать в случае перегрузки или сбоя.

- **Retry** — Повторные попытки при временных сбоях
- **Circuit Breaker** — Отключение проблемного сервиса после порога ошибок
- **Fallback** — Альтернативный ответ при недоступности сервиса
- **Bulkhead** — Изоляция компонентов для предотвращения распространения ошибки

## 🔧 Шаг 05:

### Вопрос 4

Вы сделали сервис на Go, который иногда "зависает" при обращении к внешнему API. Какой инструмент поможет ограничить время ожидания и избежать блокировки всей системы?

Подсказка: В Go для этого часто используют context.

**Ответ:**

- **Timeout через context.WithTimeout**
- Retry без ограничений
- Fallback
- Bulkhead

## Шаг 06:

### Вопрос 5

Как работает Circuit Breaker ? Расположите состояния в правильной последовательности:

1. Half-Open
2. Closed
3. Open

Подсказка: Начинаем с нормального состояния, затем при ошибках срабатывает защита, после чего можно попробовать снова.

**Ответ:**

- Closed
- Open
- Half-Open

## Шаг 07:

### Вопрос 6

У вас микросервисная архитектура. Один сервис начал падать и за собой "роняет" остальные. Какой паттерн стоит применить, чтобы сбой одного узла не останавливал всю систему?

Подсказка: Подумайте о принципе изоляции.

**Ответ:**

- **Bulkhead (отсекающий барьер)**
- Retry
- Fallback
- Timeout

## Шаг 08:

### Вопрос 7

Сопоставьте примеры поведения сервиса с правильным паттерном:

**Ответ:**

- При перегрузке сервис возвращает упрощённую версию ответа
  - Сервис блокирует запросы к упавшему компоненту
  - Клиент выполняет несколько попыток с экспоненциальной задержкой
  - Ограничение времени выполнения через context
1. Graceful degradation
  2. Circuit Breaker
  3. Retry
  4. Timeout

Подсказка: Каждая ситуация соответствует известному паттерну из курса.

- **При перегрузке сервис возвращает упрощённую версию ответа** — Graceful degradation
- **Сервис блокирует запросы к упавшему компоненту** — Circuit Breaker
- **Клиент выполняет несколько попыток с экспоненциальной задержкой** — Retry
- **Ограничение времени выполнения через context** — Timeout

## Шаг 09:

### Вопрос 8

Ваш Go-сервис обращается к внешнему API. Иногда оно полностью недоступно. Какой паттерн лучше всего применить, чтобы всё же вернуть пользователю корректный (пусть и упрощённый) результат?

Подсказка: Здесь важно предоставить запасной вариант ответа.

**Ответ:**

- **Fallback**
- Bulkhead
- Timeout
- Retry

## Шаг 10:

### Вопрос 9

Представьте, что вы реализуете устойчивый к сбоям Go-сервис . Расставьте действия в порядке реализации:

1. Добавить Circuit Breaker
2. Добавить Retry с exponential backoff
3. Ограничить время выполнения через context
4. Реализовать Fallback для критичных функций

Подсказка: Сначала защищаем от повторных ошибок и зависаний, затем добавляем запасные решения.

**Ответ:**

- Добавить Retry с exponential backoff
- Ограничить время выполнения через context
- Добавить Circuit Breaker
- Реализовать Fallback для критичных функций

## Шаг 11:

### Вопрос 10

Вы собрали высоконагруженный сервис на Go. Какие из перечисленных подходов в совокупности обеспечат наибольшую отказоустойчивость?

Подсказка: Правильный ответ объединяет несколько паттернов.

**Ответ:**

- **Circuit Breaker**
- **Retry с exponential backoff**
- **Timeout через context**
- **Fallback**
- Игнорирование ошибок

## Урок 2 - Kubernetes и Autoscaling

## Шаг 02:

### Вопрос 1

Вы разворачиваете Go-сервис в Kubernetes. Какая минимальная единица развертывания используется?

Подсказка: Она может содержать один или несколько контейнеров.

**Ответ:**

- **Pod**
- Deployment
- Service
- Namespace

### Шаг 03:

#### Вопрос 2

Сопоставьте объекты Kubernetes с их назначением:

**Ответ:**

- Deployment
  - Service
  - Namespace
  - Pod
1. Изоляция ресурсов внутри кластера
  2. Балансировка нагрузки между Pod'ами
  3. Минимальная единица развертывания
  4. Управление количеством Pod'ов и обновлениями

Подсказка: Подумайте, кто управляет, кто балансирует, кто разворачивает.

- **Deployment** — Управление количеством Pod'ов и обновлениями
- **Service** — Балансировка нагрузки между Pod'ами
- **Namespace** — Изоляция ресурсов внутри кластера
- **Pod** — Минимальная единица развертывания

### Шаг 04:

#### Вопрос 3

Ваш Go-сервис начал обрабатывать больше запросов, и нагрузка выросла. Какой объект Kubernetes позволит автоматически масштабировать количество Pod'ов на основе CPU?

Подсказка: Этот объект реагирует на метрики.

**Ответ:**

- **Horizontal Pod Autoscaler (HPA)**
- Vertical Pod Autoscaler (VPA)
- Cluster Autoscaler
- ReplicaSet

### Шаг 05:

#### Вопрос 4

Вы хотите вручную увеличить количество Pod'ов для вашего Go-сервиса. Расставьте шаги в правильном порядке:

1. Выполнить команду `kubectl scale`
2. Указать объект Deployment

3. Задать нужное количество реплик
4. Убедиться, что Pods успешно поднялись

Подсказка: Начинается всё с команды, а заканчивается проверкой результата.

**Ответ:**

- Выполнить команду `kubectl scale`
- Указать объект `Deployment`
- Задать нужное количество реплик
- Убедиться, что Pods успешно поднялись

## Шаг 06:

### Вопрос 5

Ваш сервис работает, но иногда Pod «зависает». Какой механизм Kubernetes проверяет, что Pod жив, и перезапускает его при необходимости?

Подсказка: Используется для проверки здоровья контейнера.

**Ответ:**

- **Liveness Probe**
- Readiness Probe
- Startup Probe
- Cluster Autoscaler

## Шаг 07:

### Вопрос 6

Сопоставьте Probe с его назначением:

**Ответ:**

- Liveness Probe
  - Readiness Probe
  - Startup Probe
1. Проверяет, что контейнер готов принимать трафик
  2. Перезапускает контейнер, если он завис
  3. Используется для долгого старта приложений

Подсказка: Эти проверки помогают Kubernetes понимать, как обращаться с Pod'ами.

- **Liveness Probe** — Перезапускает контейнер, если он завис
- **Readiness Probe** — Проверяет, что контейнер готов принимать трафик
- **Startup Probe** — Используется для долгого старта приложений

## Шаг 08:

### Вопрос 7

Вы раскатываете новую версию Go-сервиса и хотите минимизировать риск. Какой паттерн Kubernetes позволит сначала выпустить обновление только на часть Pod'ов?

Подсказка: Этот паттерн часто называют «канарейкой».

**Ответ:**

- **Canary release**
- Blue-Green deployment
- Rolling update

- Cluster Autoscaler

## 🔗 Шаг 09:

### Вопрос 8

Как работает Horizontal Pod Autoscaler (HPA) ? Расставьте шаги в правильной последовательности:

1. Kubernetes собирает метрики CPU/Memory
2. HPA сравнивает метрики с пороговыми значениями
3. При превышении порога увеличивается число Pod'ов
4. При снижении нагрузки Pod'ы уменьшаются до minReplicas

Подсказка: Сначала метрики, затем реакция.

**Ответ:**

- Kubernetes собирает метрики CPU/Memory
- HPA сравнивает метрики с пороговыми значениями
- При превышении порога увеличивается число Pod'ов
- При снижении нагрузки Pod'ы уменьшаются до minReplicas

## 🔗 Шаг 10:

### Вопрос 9

У вас ограниченные ресурсы в кластере. Какой механизм Kubernetes предотвращает "перегрузку" Pod'a, задавая минимальные и максимальные лимиты ресурсов?

Подсказка: Эти настройки указываются в spec → resources.

**Ответ:**

- **Resource requests & limits**
- Cluster Autoscaler
- Horizontal Pod Autoscaler
- Namespace

## 🔗 Шаг 11:

### Вопрос 10

Сопоставьте виды автоскейлинга с их назначением:

**Ответ:**

- HPA
  - VPA
  - Cluster Autoscaler
1. Автоматически изменяет ресурсы Pod'a (CPU/Memory)
  2. Добавляет или удаляет ноды в кластере
  3. Масштабирует количество Pod'ов на основе метрик

Подсказка: Один масштабирует Pod'ы, другой ресурсы, третий — сам кластер.

- **HPA** — Масштабирует количество Pod'ов на основе метрик
- **VPA** — Автоматически изменяет ресурсы Pod'a (CPU/Memory)
- **Cluster Autoscaler** — Добавляет или удаляет ноды в кластере

## Урок 3 - Load testing на уровне инфраструктуры

### Шаг 02:

#### Вопрос 1

Вы разворачиваете новый Go-сервис в Kubernetes и хотите провести нагрузочное тестирование. С чего правильнее всего начать?

Подсказка: Перед нагрузкой важно убедиться, что базовый сервис работает и доступен.

**Ответ:**

- **Проверить, что сервис корректно отвечает на базовые HTTP-запросы**
- Сразу запускать стресс-тест с 1000 RPS
- Отключить мониторинг, чтобы не мешал нагрузке
- Начать с настройки autoscaling, не тестируя сервис

### Шаг 03:

#### Вопрос 2

Какой инструмент удобнее всего использовать для быстрого генератора HTTP-запросов к Go-сервису?

Подсказка: Этот инструмент написан на Go и часто используется для быстрой проверки нагрузки.

**Ответ:**

- **hey**
- wrk2
- Apache Benchmark
- locust.io

### Шаг 04:

#### Вопрос 3

Сопоставьте вид нагрузочного тестирования с его целью.

 **Подсказка:**

Подумайте: spike — резкий рост, stress — поиск предела, soak — длительная нагрузка, chaos — отказоустойчивость.

**Ответ:**

- **Spike testing** — Резкий кратковременный рост нагрузки
- **Stress testing** — Определение точки деградации системы
- **Soak testing** — Длительная нагрузка для поиска утечек памяти
- **Chaos testing** — Проверка отказоустойчивости при сбоях

### Шаг 05:

#### Вопрос 4

Расположите шаги проведения нагрузочного тестирования Go-сервиса в Kubernetes в правильном порядке.

 **Подсказка:**

**Ответ:**

- Сначала разворачиваем сервис.
- Далее настраиваем мониторинг.

- Потом запускаем нагрузку.
- В конце анализируем метрики.
- Развернуть Go-сервис в Kubernetes
- Настроить Prometheus и Grafana
- Запустить нагрузку с hey
- Анализировать метрики и сделать выводы

## 🔮 Шаг 06:

### Вопрос 5

Во время стресс-теста latency p95 резко выросла, а CPU достигло 90%. Что правильнее всего сделать?

Подсказка: Подумайте о масштабировании подов в Kubernetes.

**Ответ:**

- **Увеличить maxReplicas в настройках HPA**
- Игнорировать проблему и продолжать тест
- Отключить мониторинг, чтобы снизить нагрузку
- Уменьшить количество пользователей без анализа

## 🔮 Шаг 07:

### Вопрос 6

Сопоставьте метрику с её назначением.

🔮 **Подсказка:**

Подумайте: RPS — скорость, latency — задержка, error rate — ошибки, resource usage — ресурсы.

**Ответ:**

- **RPS** — Пропускная способность (запросов в секунду)
- **Latency (p95)** — Средняя задержка обработки запросов
- **Error rate** — Процент отказов
- **Resource usage** — Загрузка CPU и памяти

## 🔮 Шаг 08:

### Вопрос 7

Расположите шаги анализа результатов нагрузочного теста.

🔮 **Подсказка:**

**Ответ:**

- Сначала собираем метрики.
- Затем ищем узкие места.
- После этого формируем гипотезы улучшения.
- В конце вносим изменения и повторяем тест.
- Собрать метрики (CPU, memory, latency)
- Определить узкие места
- Предложить улучшения (настройка HPA, оптимизация кода)
- Применить изменения и перезапустить нагрузку

## Шаг 09:

### Вопрос 8

Вы заметили рост количества 5xx ошибок во время soak-теста. Какое действие будет наиболее верным?

Подсказка: Ошибки 5xx обычно связаны с таймаутами или зависимыми сервисами.

**Ответ:**

- **Проверить таймауты и Circuit Breaker**
- Отключить все проверки готовности Pod'ов
- Просто увеличить число пользователей
- Игнорировать ошибки, если их меньше 50%

## Шаг 10:

### Вопрос 9

Сопоставьте инструмент с его применением в нагрузочном тестировании.

 **Подсказка:**

Вспомните: wrk/wrk2 — быстрый http тест, k6 — сценарии, locust — распределённый тест, Prometheus/Grafana — мониторинг.

**Ответ:**

- **wrk / wrk2** — Высокопроизводительное HTTP тестирование
- **k6** — Сценарии нагрузки с отчётами
- **locust.io** — Распределённое нагрузочное тестирование
- **Prometheus + Grafana** — Сбор и визуализация метрик

## Шаг 11:

### Вопрос 10

Упорядочьте шаги при настройке авто-масштабирования (HPA) и проведении итогового нагрузочного теста.

 **Подсказка:**

**Ответ:**

- Сначала задаём минимальное и максимальное число реплик.
- Потом задаём метрики (CPU/Memory).
- Запускаем нагрузку.
- Смотрим, как HPA масштабирует Pod'ы.
- Задать minReplicas и maxReplicas
- Настроить таргетные метрики CPU/Memory
- Запустить нагрузку hey
- Проверить реакцию HPA на нагрузку

## Урок 4 - Multi-datacenter архитектура

## Шаг 02:

### Вопрос 1

Вы разрабатываете сервис, который обслуживает пользователей из Европы и США. Чтобы минимизировать задержки, вы решаете использовать Multi-datacenter архитектуру. Какой из вариантов лучше всего отражает её основную цель?

Подсказка: Вспомните, зачем компании распределяют сервисы по разным регионам.

**Ответ:**

- **Снижение задержек и повышение отказоустойчивости**
- Экономия на аренде серверов
- Уменьшение количества функций в сервисе
- Сокращение числа пользователей

### Шаг 03:

#### **Вопрос 2**

Представьте, что вы настраиваете новый дата-центр. Расположите шаги настройки в правильном порядке:

1. Настройка балансировщика между дата-центрами
2. Развёртывание сервисов в новом дата-центре
3. Добавление репликации данных
4. Тестирование отказоустойчивости

 **Подсказка:**

Сначала сервисы должны появиться в дата-центре, потом данные, потом балансировка и только потом проверка.

**Ответ:**

- Развёртывание сервисов в новом дата-центре
- Добавление репликации данных
- Настройка балансировщика между дата-центрами
- Тестирование отказоустойчивости

### Шаг 04:

#### **Вопрос 3**

Сопоставьте тип архитектуры с его описанием.

**Ответ:**

- Active-Active
  - Active-Passive
  - Geo-sharding
  - Global Load Balancer
1. Каждый дата-центр обслуживает только свой регион или часть данных.
  2. Один дата-центр принимает записи, остальные работают только на чтение.
  3. Все дата-центры могут одновременно обрабатывать записи.
  4. DNS или L7-балансировщик распределяет трафик по регионам.

Подсказка: Вспомните разницу между активными и пассивными режимами.

- **Active-Active** — Все дата-центры могут одновременно обрабатывать записи.
- **Active-Passive** — Один дата-центр принимает записи, остальные работают только на чтение.
- **Geo-sharding** — Каждый дата-центр обслуживает только свой регион или часть данных.
- **Global Load Balancer** — DNS или L7-балансировщик распределяет трафик по регионам.

## 🔗 Шаг 05:

### Вопрос 4

Ваша команда обсуждает стратегию репликации между дата-центрами. Вы хотите сильную консистентность, но боитесь задержек. Какой подход к репликации вы выберете, если приоритет именно в консистентности?

Подсказка: Вспомните trade-off между latency и согласованностью.

**Ответ:**

- **Синхронная репликация**
- Асинхронная репликация
- Отсутствие репликации
- Репликация по cron раз в сутки

## 🔗 Шаг 06:

### Вопрос 5

При сбое одного дата-центра важно корректно выполнить failover. Расположите шаги в правильной последовательности:

1. Определение падения дата-центра (health check)
2. Перенаправление трафика на другой дата-центр
3. Синхронизация данных с резервного DC
4. Уведомление команды о сбое

🔗 **Подсказка:**

Сначала нужно понять, что DC упал, потом переключить трафик, потом синхронизировать данные и только потом уведомлять.

**Ответ:**

- Определение падения дата-центра (health check)
- Перенаправление трафика на другой дата-центр
- Синхронизация данных с резервного DC
- Уведомление команды о сбое

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте тип очереди/кэша с его преимуществом в Multi-datacenter архитектуре.

**Ответ:**

- Kafka
  - Redis Cluster
  - NATS
  - Memcached
1. Простое распределение ключей и высокая скорость кэша, но без персистентности.
  2. Надёжная очередь с репликацией событий между DC.
  3. Лёгкий брокер сообщений, быстрый и простой для интеграции.
  4. Распределённый кэш с репликацией и отказоустойчивостью.

Подсказка: Вспомните: Kafka = события, Redis = кэш с репликами, NATS = лёгкий брокер, Memcached = быстрый кэш без сохранности.

- **Kafka** — Надёжная очередь с репликацией событий между DC.

- **Redis Cluster** — Распределённый кэш с репликацией и отказоустойчивостью.
- **NATS** — Лёгкий брокер сообщений, быстрый и простой для интеграции.
- **Memcached** — Простое распределение ключей и высокая скорость кэша, но без персистентности.

## 🔗 Шаг 08:

### Вопрос 7

Вы пишете сервис на Go, который должен перенаправлять пользователей в ближайший дата-центр. Какой параметр HTTP-запроса чаще всего помогает определить регион пользователя?

Подсказка: Вспомните, как в примере с Go-сервисом выбирался дата-центр.

**Ответ:**

- **Заголовок X-User-Region**
- User-Agent
- Content-Type
- Accept-Encoding

## 🔗 Шаг 09:

### Вопрос 8

Вы реализуете систему асинхронной записи данных в реплику. Расположите шаги выполнения операции:

1. Получение запроса на запись
2. Сохранение данных в основной DC
3. Отправка события в очередь
4. Обработка события и запись в реплику

🔗 **Подсказка:**

Сначала принимаем запрос, потом пишем в главный DC, потом кладём событие в очередь, а уже потом реплика записывает данные.

**Ответ:**

- Получение запроса на запись
- Сохранение данных в основной DC
- Отправка события в очередь
- Обработка события и запись в реплику

## 🔗 Шаг 10:

### Вопрос 9

Сопоставьте best practice с её назначением.

**Ответ:**

- Идемпотентные операции
  - Мониторинг задержек репликации
  - Настройка retries и timeouts
  - Логирование событий по DC
1. Помогает видеть расхождения между дата-центрами.
  2. Гарантирует, что повторная запись не создаст дубликатов.
  3. Даёт понимание состояния каждого DC и его нагрузки.
  4. Позволяет избежать долгих подвисаний при проблемах с сетью.

Подсказка: Каждая практика — это защита от конкретной проблемы: дубликаты, задержки, сбои сети, непрозрачность.

- **Идемпотентные операции** — Гарантирует, что повторная запись не создаст дубликатов.
- **Мониторинг задержек репликации** — Помогает видеть расхождения между дата-центрами.
- **Настройка retries и timeouts** — Позволяет избежать долгих подвисаний при проблемах с сетью.
- **Логирование событий по DC** — Даёт понимание состояния каждого DC и его нагрузки.

## Шаг 11:

### Вопрос 10

В ходе нагрузочного тестирования выяснилось, что при росте трафика один из дата-центров перегружается быстрее других. Какое решение позволит равномерно распределить нагрузку?

Подсказка: Вспомните про глобальные балансировщики и распределение трафика.

**Ответ:**

- **Использование глобального балансировщика (DNS/L7)**
- Отключение половины пользователей
- Увеличение задержек искусственно
- Удаление репликации

## Модуль 6 - Мониторинг и тестирование

### Урок 1 - Мониторинг и профилирование

## Шаг 02:

### Вопрос 1

Вы запускаете новый Go-сервис. Вам важно знать, что он вообще работает. Какой первый и самый простой способ мониторинга стоит внедрить?

Подсказка: Начните с базового показателя, который отражает "жив ли сервис".

**Ответ:**

- **Метрика количества HTTP-запросов (http\_requests\_total)**
- Heap profile
- Mutex profile
- Tracing всех goroutine

## Шаг 03:

### Вопрос 2

В продакшне сервис стал отвечать медленно. Какие метрики помогут понять, в какой момент пользователи замечают деградацию?

Подсказка: Среднее значение не отражает реальных проблем, нужно смотреть глубже.

**Ответ:**

- Среднее время ответа
- **Latency p95 и p99**
- Количество Pod'ов
- Размер логов

## 🔗 Шаг 04:

### Вопрос 3

Сопоставьте тип профиля Go с тем, какую проблему он помогает выявить:

#### 🔗 Подсказка:

Каждый профиль отвечает за свою "боль": CPU — про нагрузку, Heap — про память, Goroutine — про блокировки.

#### Ответ:

- **CPU profile** — Какие функции грузят процессор
- **Heap profile** — Куда утекает память
- **Goroutine profile** — Есть ли зависания и конкуренция
- **Mutex profile** — Блокировки на мьютексах

## 🔗 Шаг 05:

### Вопрос 4

Вы заметили рост времени отклика сервиса. Расставьте действия в правильном порядке:

#### Ответ:

- Собрать pprof профиль
- Сначала проверить метрики latency
- Найти проблемный участок кода
- Внести исправления и проверить снова

Подсказка: Сначала видим симптомы, потом измеряем, ищем проблему и только после этого чиним.

- Сначала проверить метрики latency
- Собрать pprof профиль
- Найти проблемный участок кода
- Внести исправления и проверить снова

## 🔗 Шаг 06:

### Вопрос 5

DevOps сообщает: CPU сервера загружен на 95%, но сервис всё ещё быстрый. Что это значит?

Подсказка: Смотрите на конечный результат для пользователей, а не только на сырые числа.

#### Ответ:

- Сервис не справляется с нагрузкой
- **Высокая утилизация CPU допустима, если latency и error rate в норме**
- Необходимо немедленно выключить сервис
- Нужно увеличить количество Pod'ов, даже если нет ошибок

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте инструменты мониторинга/профилирования с их основным применением:

Подсказка: Prometheus — про метрики, Grafana — про графики, Jaeger — про трассы.

#### Ответ:

- **Prometheus** — Сбор метрик

- **Grafana** — Визуализация и дашборды
- **Jaeger** — Распределённый трейсинг
- **pprof** — Профилирование Go-программ

## 🔗 Шаг 08:

### Вопрос 7

Вы добавили Prometheus-метрики в сервис. Какую из метрик стоит использовать для отслеживания количества ошибок?

Подсказка: Ошибки считаются через отдельный counter.

**Ответ:**

- http\_requests\_total
- **http\_requests\_errors\_total**
- process\_memory\_bytes
- goroutines\_total

## 🔗 Шаг 09:

### Вопрос 8

Представьте: вы внедряете алертинг на latency. Расставьте шаги в правильном порядке:

**Ответ:**

- Создать правило алерта в Prometheus
- Определить метрику (например, p95 latency)
- Задать порог (например, >200ms)
- Протестировать, что алерт срабатывает при нагрузке

Подсказка: Сначала определяем, что измеряем, потом задаём порог, настраиваем правило и проверяем.

- Определить метрику (например, p95 latency)
- Задать порог (например, >200ms)
- Создать правило алерта в Prometheus
- Протестировать, что алерт срабатывает при нагрузке

## 🔗 Шаг 10:

### Вопрос 9

В профиле goroutine вы видите тысячи зависших горутин в состоянии chan send . Что это может значить?

Подсказка: Подумайте о блокировках каналов.

**Ответ:**

- **Отправка в канал без активного получателя**
- Нехватка CPU
- Проблема с памятью Heap
- Ошибка компилятора Go

## 🔗 Шаг 11:

### Вопрос 10

Сопоставьте метрику с тем, какую задачу она помогает решать:

Подсказка: RPS — про нагрузку, error rate — про стабильность, CPU/memory — про ресурсы.

**Ответ:**

- **RPS** — Понимание текущей нагрузки
- **Latency p95** — Видно деградацию скорости
- **Error rate** — Контроль стабильности
- **CPU usage** — Утилизация ресурсов
- **Memory usage** — Обнаружение утечек памяти

## **Урок 2 - Нагрузочное и хаос-тестирование**

### **Шаг 02:**

#### **Вопрос 1**

Вы только что развернули свой первый Go-сервис и хотите проверить его производительность. Какой первый шаг в нагрузочном тестировании стоит сделать?

Подсказка: Нужно зафиксировать нормальные показатели системы, прежде чем увеличивать нагрузку.

**Ответ:**

- **Снять baseline метрик при стандартной нагрузке**
- Сразу запустить стресс-тест на максимальной нагрузке
- Удалить Pod в Kubernetes, чтобы проверить отказоустойчивость
- Вручную перегрузить CPU с помощью бесконечного цикла

### **Шаг 03:**

#### **Вопрос 2**

При нагрузочном тестировании Go-сервиса вы заметили, что задержка ответов выросла с 50ms до 500ms при росте нагрузки. Что это чаще всего означает?

Подсказка: В таких случаях речь идёт о появлении узкого места.

**Ответ:**

- **Появился bottleneck в системе**
- Увеличилась скорость сети
- Все Pod'ы автоматически перезапустились
- Метрики p95 всегда равны нулю

### **Шаг 04:**

#### **Вопрос 3**

Какой вид нагрузочного тестирования поможет выявить деградацию работы системы при длительной работе под нагрузкой?

Подсказка: Этот вид теста проверяет утечки памяти и медленное падение производительности.

**Ответ:**

- **Soak testing (продолжительная нагрузка)**
- Spike testing (резкие пики нагрузки)
- Stress testing (нагрузка до отказа)
- Unit testing (юнит-тесты функций)

### **Шаг 05:**

#### **Вопрос 4**

Расположите шаги проведения нагрузочного тестирования в правильном порядке:

 **Подсказка:**

Сначала фиксируем baseline, потом увеличиваем нагрузку, далее добавляем хаос, а в конце собираем метрики.

**Ответ:**

- Снять baseline метрик
- Постепенно увеличивать нагрузку
- Добавить элементы хаоса (например, удалить Pod)
- Собрать и проанализировать метрики

 **Шаг 06:**

**Вопрос 5**

Вы проводите Spike-тестирование. Что именно вы проверяете?

Подсказка: Важна реакция системы на резкие пики нагрузки.

**Ответ:**

- **Устойчивость системы к кратковременным резким всплескам запросов**
- Как система ведёт себя без нагрузки
- Как быстро растёт размер базы данных
- Что происходит при остановке всех Pod'ов навсегда

 **Шаг 07:**

**Вопрос 6**

В хаос-тестировании вы удаляете один Pod в Kubernetes. Какое поведение будет правильным для устойчивой highload-системы?

Подсказка: Подумайте про HPA и самовосстановление.

**Ответ:**

- **Kubernetes создаёт новый Pod, нагрузка перераспределяется**
- Весь сервис падает и перестаёт отвечать
- CPU и память автоматически становятся бесконечными
- Go-код сам создаёт Pod без участия Kubernetes

 **Шаг 08:**

**Вопрос 7**

Расположите типы нагрузочного тестирования в порядке от самого мягкого до самого экстремального:

 **Подсказка:**

Сначала проверка на длительной нагрузке, затем пики, и только потом тестирование до полного отказа.

**Ответ:**

- Soak testing
- Spike testing
- Stress testing

 **Шаг 09:**

**Вопрос 8**

Сопоставьте метрику с её назначением:

🔗 **Подсказка:**

Подумайте, какая метрика показывает скорость, какая надёжность, а какая ошибки.

**Ответ:**

- **RPS (Requests per Second)** — Количество обработанных запросов в секунду
- **Latency p95** — Время ответа сервиса для 95% запросов
- **Error rate** — Процент запросов, завершившихся с ошибкой

🔗 **Шаг 10:**

### Вопрос 9

Сопоставьте инструмент с его основным применением в нагрузочном или хаос-тестировании:

🔗 **Подсказка:**

hey используется для генерации трафика, Chaos Monkey вносит сбои, а Grafana помогает видеть картину.

**Ответ:**

- **hey** — Генерация нагрузки на сервис
- **Chaos Monkey** — Внесение случайных сбоев
- **Grafana** — Мониторинг и визуализация метрик

🔗 **Шаг 11:**

### Вопрос 10

Сопоставьте ситуацию с правильным типом тестирования:

🔗 **Подсказка:**

Вспомните: пики нагрузки → Spike, длительное использование → Soak, перегрузка до отказа → Stress.

**Ответ:**

- **Резкий рост числа запросов во время распродажи** — Spike testing
- **Работа системы под средней нагрузкой в течение 24 часов** — Soak testing
- **Постепенное увеличение нагрузки до полного отказа сервиса** — Stress testing

## 📖 Урок 3 - Трассировка (tracing)

🔗 **Шаг 02:**

### Вопрос 1

Вы только подключили OpenTelemetry к своему Go-сервису. Какую задачу трассировка решает в первую очередь?

Подсказка: Подумайте: метрики показывают общую картину, а трассировка помогает понять детали.

**Ответ:**

- **Позволяет увидеть путь запроса через разные сервисы**
- Ускоряет компиляцию Go-кода
- Заменяет логирование полностью
- Уменьшает потребление памяти

## 🔗 Шаг 03:

### Вопрос 2

Сопоставьте основные понятия трассировки с их определениями:

Подсказка: Вспомните: trace — это целое путешествие запроса, span — его шаг.

**Ответ:**

- **Trace** — Совокупность связанных span'ов, представляющих путь одного запроса.
- **Span** — Единица работы — например, обработка HTTP-запроса или вызов БД.
- **Context propagation** — Передача trace-id и span-id между сервисами.
- **Sampling** — Отбор части запросов для трассировки, чтобы снизить нагрузку.

## 🔗 Шаг 04:

### Вопрос 3

Ваш сервис внезапно стал отвечать медленнее. Метрики RPS и latency не показывают конкретику. Что поможет быстрее всего найти узкое место?

Подсказка: Подумайте, что даст детализацию по каждому вызову внутри запроса.

**Ответ:**

- **Распределённая трассировка**
- Сбор дампов памяти
- Увеличение логирования ошибок
- Перезапуск сервера

## 🔗 Шаг 05:

### Вопрос 4

Какой порядок действий при добавлении трассировки в Go-сервис?

Подсказка: Сначала подключают библиотеку, затем настраивают экспорт, потом создают спаны.

**Ответ:**

- Подключить OpenTelemetry SDK
- Создать TracerProvider и экспортёр (Jaeger/Zipkin)
- Добавить спаны в ключевые операции
- Передавать контекст между сервисами

## 🔗 Шаг 06:

### Вопрос 5

В трассировке запроса видно, что span для обращения к базе данных занимает слишком много времени. Что это означает?

Подсказка: Смотрите не только на сам сервис, но и на внешние вызовы.

**Ответ:**

- **База данных отвечает медленно, это узкое место**
- Код трассировки работает неправильно
- OpenTelemetry замедляет приложение
- Это нормальная ситуация и не требует внимания

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте инструмент и его основную роль в observability:

Подсказка: Prometheus собирает метрики, Jaeger и Zipkin показывают трассы, Grafana строит красивые графики.

**Ответ:**

- **OpenTelemetry** — Стандарт сбора метрик, логов и трассировок.
- **Jaeger** — Визуализация распределённых трасс.
- **Zipkin** — Анализ трасс запросов, альтернатива Jaeger.
- **Prometheus + Grafana** — Сбор метрик и построение дашбордов.

#### Шаг 08:

##### Вопрос 7

Вы настроили `sampling=100%` и заметили, что сервис стал потреблять слишком много ресурсов. Как лучше всего поступить?

Подсказка: Трассировать можно не каждый запрос.

**Ответ:**

- **Уменьшить `sampling`, например до 10%**
- Полностью отключить трассировку
- Оставить как есть
- Перейти на логирование вместо трассировки

#### Шаг 09:

##### Вопрос 8

Расположите шаги анализа трассы запроса в правильном порядке:

Подсказка: Сначала открываем трассу, потом ищем самый долгий span, затем проверяем цепочку вызовов и делаем выводы.

**Ответ:**

- Открыть трассу в Jaeger/Zipkin
- Найти span с наибольшей задержкой
- Проследить зависимые вызовы
- Определить узкое место и сформулировать гипотезу

#### Шаг 10:

##### Вопрос 9

Вы заметили, что в трассах теряется связь между сервисами: `trace-id` не передаётся дальше. Что нужно сделать?

Подсказка: Вспомните про context propagation.

**Ответ:**

- **Передавать context с `trace-id` при вызовах между сервисами**
- Включить больше логов
- Увеличить `sampling` до 100%
- Переписать сервисы на другом языке

#### Шаг 11:

##### Вопрос 10

Сопоставьте проблему и решение при работе с трассировкой:

Подсказка: Вспомните: sampling спасает от нагрузки, спаны помогают искать узкие места, context связывает вызовы.

**Ответ:**

- **Большая нагрузка от трассировки** — Использовать sampling.
- **Потеря цепочки вызовов между сервисами** — Правильно передавать context.
- **Долгие ответы базы данных** — Проанализировать span DB-запроса.
- **Сложно понять, где задержка в запросе** — Смотреть на последовательность span'ов.

## Урок 4 - Chaos engineering глубже

### Шаг 02:

#### Вопрос 1

Вы разрабатываете высоконагруженный сервис на Go и хотите начать изучать Chaos Engineering. С чего лучше всего начать?

Подсказка: Chaos Engineering строится на принципе «малых доз» — начинаем с простых экспериментов и наращиваем сложность.

**Ответ:**

- **Проверить реакцию системы на отключение одного Pod'a**
- Сразу симулировать падение целого дата-центра
- Устроить стресс-тест без метрик
- Выключить все сервисы одновременно

### Шаг 03:

#### Вопрос 2

В продвинутых хаос-экспериментах важна гипотеза. Какой пример правильной гипотезы?

Подсказка: Гипотеза должна описывать ожидаемое поведение системы под хаосом.

**Ответ:**

- **При задержке сети в 500ms Circuit Breaker должен открыться и не допустить cascade failure**
- Мы надеемся, что всё будет работать как-нибудь
- Хаос приведёт к интересным результатам
- Система точно упадёт при любом сбое

### Шаг 04:

#### Вопрос 3

Расположите шаги хаос-эксперимента в правильном порядке:

1. Сбор метрик и логов
2. Определение гипотезы
3. Запуск хаос-эксперимента
4. Настройка мониторинга и алертов

### Подсказка:

Сначала формируем гипотезу, затем обеспечиваем наблюдаемость, запускаем эксперимент и в конце анализируем результаты.

**Ответ:**

- Определение гипотезы

- Настройка мониторинга и алертов
- Запуск хаос-эксперимента
- Сбор метрик и логов

#### 🔮 Шаг 05:

##### Вопрос 4

Сопоставьте тип хаоса с его примером.

**Ответ:**

- Network chaos
  - Resource chaos
  - Service chaos
  - Dependency chaos
1. Выключение случайного Pod'a
  2. Потеря пакетов в сети
  3. Spike CPU до 100%
  4. Недоступность базы данных

#### 🔮 Подсказка:

Подумайте, к чему относится каждая проблема — сеть, ресурсы, сервис или внешняя зависимость.

- **Network chaos** — Потеря пакетов в сети
- **Resource chaos** — Spike CPU до 100%
- **Service chaos** — Выключение случайного Pod'a
- **Dependency chaos** — Недоступность базы данных

#### 🔮 Шаг 06:

##### Вопрос 5

Вы запустили эксперимент с задержкой сети 500ms. Что вы должны увидеть, если HPA и Circuit Breaker настроены правильно?

Подсказка: Ожидаемое поведение системы — адаптация, а не полный отказ.

**Ответ:**

- **Рост реплик сервиса и открытие Circuit Breaker**
- Полный отказ системы
- Игнорирование задержки без реакции
- Непредсказуемые ошибки без метрик

#### 🔮 Шаг 07:

##### Вопрос 6

Расположите шаги при настройке LitmusChaos в Kubernetes:

1. Создать ChaosExperiment
2. Установить LitmusChaos оператор
3. Запустить эксперимент
4. Проверить метрики и результаты

#### 🔮 Подсказка:

Сначала нужно установить сам инструмент, затем определить эксперимент, запустить его и наконец проанализировать.

**Ответ:**

- Установить LitmusChaos оператор
- Создать ChaosExperiment
- Запустить эксперимент
- Проверить метрики и результаты

 **Шаг 08:**

**Вопрос 7**

Сопоставьте инструмент с его особенностью:

**Ответ:**

- LitmusChaos
  - Chaos Mesh
  - Gremlin
  - Kubectl + scripts
1. Коммерческий инструмент с визуальным управлением
  2. Kubernetes-native фреймворк с большим набором сценариев
  3. Базовая автоматизация через скрипты
  4. Ориентирован на cloud-native хаос

 **Подсказка:**

Вспомните, какой инструмент является коммерческим, а какой open-source.

- **LitmusChaos** — Kubernetes-native фреймворк с большим набором сценариев
- **Chaos Mesh** — Ориентирован на cloud-native хаос
- **Gremlin** — Коммерческий инструмент с визуальным управлением
- **Kubectl + scripts** — Базовая автоматизация через скрипты

 **Шаг 09:**

**Вопрос 8**

При хаос-тестировании важно ограничивать blast radius. Что это значит?

Подсказка: Blast radius — это область воздействия хаоса.

**Ответ:**

- **Запускать эксперимент на ограниченном числе Pod'ов или сервисов**
- Запускать хаос только ночью
- Всегда ломать всю систему
- Случайно выключать весь кластер

 **Шаг 10:**

**Вопрос 9**

Вы готовите комбинированный эксперимент: нагрузочное тестирование + хаос. Расположите действия:

1. Запустить нагрузочный инструмент (hey/wrk)
2. Определить сценарий хаоса
3. Запустить хаос-эксперимент

#### 4. Собрать метрики и трассировки

##### **Подсказка:**

Нагрузка должна идти одновременно с хаосом, а анализ выполняется в конце.

##### **Ответ:**

- Определить сценарий хаоса
- Запустить нагрузочный инструмент (hey/wrk)
- Запустить хаос-эксперимент
- Собрать метрики и трассировки

##### **Шаг 11:**

##### **Вопрос 10**

Сопоставьте метрику с тем, что она показывает при хаос-экспериментах:

##### **Ответ:**

- Latency
  - Error rate
  - RPS
  - HPA metrics
1. Частота ошибок при обращении к сервису
  2. Задержка ответа
  3. Количество запросов в секунду
  4. Реакция системы масштабированием

##### **Подсказка:**

Эти метрики обычно смотрят в Grafana и Jaeger после хаоса.

- **Latency** — Задержка ответа
- **Error rate** — Частота ошибок при обращении к сервису
- **RPS** — Количество запросов в секунду
- **HPA metrics** — Реакция системы масштабированием

## **Модуль 7 - Итоговый проект**

### **Урок 1 - Практическая реализация Highload-сервиса**

##### **Шаг 02:**

##### **Вопрос 1**

Представьте: вы начинаете писать первый высоконагруженный сервис на Go. С чего важно начать?

Подсказка: Прежде чем оптимизировать, нужно понять базовые цели highload-систем.

##### **Ответ:**

- **Определить требования: скорость, масштабируемость, отказоустойчивость**
- Сразу добавить Redis и Nginx
- Сразу подключить Prometheus
- Начать с написания тестов нагрузки

## 🔗 Шаг 03:

### Вопрос 2

Сопоставьте примеры сервисов с характеристиками highload-систем.

Подсказка: Подумайте, где важнее миллисекунды, а где — хранение огромных объёмов данных.

**Ответ:**

- **Мессенджер** — Требуется минимальной задержки при обмене сообщениями
- **Социальная сеть** — Нужна масштабируемость при росте числа пользователей
- **Платёжная система** — Требуется высокой отказоустойчивости и безопасности
- **Видеохостинг** — Критична скорость доставки контента

## 🔗 Шаг 04:

### Вопрос 3

Вы работаете над масштабированием сервиса. Чем горизонтальное масштабирование отличается от вертикального?

Подсказка: Один подход — усиливать один сервер, другой — добавлять новые.

**Ответ:**

- **Горизонтальное** — это добавление новых серверов, вертикальное — увеличение ресурсов одного сервера
- **Горизонтальное** — это только кеширование, вертикальное — только база данных
- **Горизонтальное** — это про API, вертикальное — про микросервисы
- Они ничем не отличаются

## 🔗 Шаг 05:

### Вопрос 4

Представьте: вам нужно развернуть несколько копий Go-сервиса с балансировкой. Расставьте шаги в правильном порядке.

Подсказка: Сначала запускаем сервисы, потом подключаем балансировщик.

**Ответ:**

- Запустить несколько экземпляров Go-сервиса на разных портах
- Настроить Nginx upstream с серверами
- Прописать location с proxy\_pass
- Протестировать нагрузкой (ab или hey)

## 🔗 Шаг 06:

### Вопрос 5

В коде на Go вы хотите ускорить работу за счёт кеширования. Какую технологию чаще всего выбирают?

Подсказка: Она умеет хранить ключ-значение в памяти и быстро отдавать данные.

**Ответ:**

- **Redis**
- MySQL
- Elasticsearch
- Kafka

## 🔮 Шаг 07:

### Вопрос 6

Сопоставьте инструмент с его назначением в highload-сервисах.

Подсказка: Балансировка ≠ Мониторинг ≠ Кэш.

**Ответ:**

- **Redis** — Хранение данных в памяти для ускорения доступа
- **PostgreSQL** — Основная база данных
- **Nginx** — Балансировка нагрузки
- **Prometheus** — Сбор и хранение метрик

## 🔮 Шаг 08:

### Вопрос 7

Ваш сервис на Go внезапно стал работать медленнее под нагрузкой. С чего лучше начать поиск проблемы?

Подсказка: Ищем узкие места: CPU, память, база.

**Ответ:**

- **Запустить нагрузочное тестирование и посмотреть метрики**
- Сразу переписать сервис на другой язык
- Добавить больше логов без анализа
- Увеличить количество серверов без проверки причин

## 🔮 Шаг 09:

### Вопрос 8

Вы внедряете кэширование в Go-сервисе. Какой порядок шагов верный?

Подсказка: Сначала проверяем кэш, потом — источник данных.

**Ответ:**

- Получить значение из Redis
- **Если ключ не найден** — запросить из базы
- Сохранить в Redis
- Вернуть результат пользователю

## 🔮 Шаг 10:

### Вопрос 9

Сопоставьте метрику мониторинга с её значением для highload-сервиса.

Подсказка: Метрики помогают найти узкие места.

**Ответ:**

- **Latency (задержка)** — Время ответа сервиса
- **RPS (Requests per second)** — Количество запросов в секунду
- **Error rate** — Доля неуспешных запросов
- **CPU usage** — Нагрузка на процессор

## 🔮 Шаг 11:

### Вопрос 10

Сервис выдержал нагрузку, но база данных всё равно тормозит. Что чаще всего применяют для разгрузки базы?

Подсказка: Вспомните, как часто мы читаем одни и те же данные.

**Ответ:**

- **Использовать кэширование (Redis, Memcached)**
- Добавить больше логов
- Снизить количество пользователей
- Переписать сервис на C++

## Урок 2 - CI/CD для highload-сервисов

### Шаг 02:

#### Вопрос 1

Почему в высоконагруженных системах ручное развертывание считается плохой практикой?

Подсказка: Подумайте о рисках ошибок и времени простоя.

**Ответ:**

- **Высокий риск ошибок и простоев при обновлении вручную**
- Так быстрее и удобнее для команды
- Ручное развертывание обеспечивает лучшую безопасность
- Автоматизация занимает слишком много времени

### Шаг 03:

#### Вопрос 2

Сопоставьте компоненты CI/CD с их назначением.

Подсказка: CI связано со сборкой и тестированием, а CD с выкатыванием на окружения.

**Ответ:**

- **Continuous Integration** — Автоматическая сборка и тестирование кода
- **Continuous Delivery** — Автоматическая подготовка к развертыванию
- **Continuous Deployment** — Автоматическое развертывание в production
- **Pipeline** — Набор последовательных шагов автоматизации

### Шаг 04:

#### Вопрос 3

Какое преимущество даёт Docker при настройке CI/CD для Go-сервиса?

Подсказка: Вспомните про переносимость окружений.

**Ответ:**

- **Одинаковое окружение для сборки и запуска на разных серверах**
- Docker ускоряет работу кода Go в 10 раз
- Docker заменяет необходимость писать тесты
- Docker нужен только для frontend-приложений

### Шаг 05:

#### Вопрос 4

Расположите шаги сборки Go-сервиса с использованием Docker в правильном порядке:

Подсказка: Сначала скачиваем зависимости, затем собираем бинарник, после — создаём финальный образ.

**Ответ:**

- Скачивание зависимостей (go mod download)
- Сборка бинарного файла (go build)
- Создание финального Docker-образа
- Запуск контейнера из образа

## 🔗 Шаг 06:

### Вопрос 5

Для чего в GitHub Actions указывают шаг go test ./... перед сборкой Docker-образа?

Подсказка: Подумайте о том, что произойдёт, если тесты будут выполняться уже в production.

**Ответ:**

- **Чтобы убедиться, что изменения не сломали функционал**
- Чтобы ускорить процесс сборки
- Чтобы проверить Dockerfile
- Чтобы проверить скорость работы сервера

## 🔗 Шаг 07:

### Вопрос 6

Сопоставьте инструменты с их ролью в CI/CD pipeline.

Подсказка: Jenkins и GitHub Actions — оркестрация, ArgoCD — деплоймент, Prometheus — мониторинг.

**Ответ:**

- **Jenkins** — Оркестрация CI/CD процессов
- **GitHub Actions** — Автоматизация сборки и тестов прямо в репозитории
- **ArgoCD** — Деплоймент в Kubernetes
- **Prometheus** — Мониторинг метрик сервиса

## 🔗 Шаг 08:

### Вопрос 7

Какой подход в highload-системах позволяет минимизировать простой при релизе новой версии?

Подсказка: Два окружения, переключение трафика.

**Ответ:**

- **Blue-Green Deployment**
- Монолитное обновление
- Ручное выкатывание
- Перезапуск всех серверов одновременно

## 🔗 Шаг 09:

### Вопрос 8

Расположите шаги Blue-Green деплоймента в правильной последовательности:

Подсказка: Сначала разворачивается новая версия, потом проверка, и только потом переключение трафика.

**Ответ:**

- Развернуть новую версию в неактивное окружение
- Проверить корректность работы новой версии
- Переключить трафик на новое окружение
- Отключить старое окружение

#### Шаг 10:

##### **Вопрос 9**

Что такое канареечный релиз в контексте highload-сервисов?

Подсказка: Вспомните про постепенное выкатывание изменений.

**Ответ:**

- **Обновление сначала для небольшой части пользователей с последующим расширением**
- Одновременное обновление всех серверов
- Тестирование только в staging-среде
- Выкат новой версии только ночью

#### Шаг 11:

##### **Вопрос 10**

Сопоставьте действия в CI/CD с их назначением при откате версии.

Подсказка: Откат — это автоматизированный возврат к предыдущему состоянию.

**Ответ:**

- **Мониторинг метрик после релиза** — Определение, работает ли новая версия корректно
- **Алертинг** — Уведомление команды о проблемах
- **Rollback** — Автоматический возврат к предыдущей версии
- **Версионирование Docker-образов** — Возможность быстро развернуть стабильную версию

### Урок 3 - Финальное нагрузочное тестирование

#### Шаг 02:

##### **Вопрос 1**

Зачем нужно нагрузочное тестирование перед релизом highload-сервиса?

Подсказка: Подумайте о проверке производительности и устойчивости системы.

**Ответ:**

- **Чтобы проверить производительность и выявить узкие места**
- Чтобы заменить unit-тесты
- Чтобы снизить скорость отклика сервиса
- Чтобы проверить работу UI

#### Шаг 03:

##### **Вопрос 2**

Сопоставьте метрики с их описанием.

Подсказка: Throughput — это скорость, Latency — время отклика.

**Ответ:**

- **Throughput** — Количество запросов в секунду
- **Latency** — Время отклика сервиса
- **Error Rate** — Процент ошибок/отказов
- **Resource Usage** — Использование CPU, RAM, диска и сети

#### 🔗 Шаг 04:

##### Вопрос 3

Какой инструмент лучше всего подходит для простого HTTP-нагрузочного теста на Go-сервисе?

Подсказка: Ищите лёгкий CLI-инструмент для быстрой проверки сервиса.

**Ответ:**

- **hey**
- Docker Compose
- Kubernetes
- Go Build

#### 🔗 Шаг 05:

##### Вопрос 4

Расположите шаги простого нагрузочного теста с hey в правильном порядке:

Подсказка: Сначала запускаем команду, затем собираем результаты, потом анализируем метрики.

**Ответ:**

- Запуск hey с нужным числом пользователей и временем теста
- Сбор метрик: latency, error rate, throughput
- Анализ узких мест и выявление проблем
- Корректировка параметров сервиса или инфраструктуры

#### 🔗 Шаг 06:

##### Вопрос 5

Что позволяет выявить стресс-тестирование highload-сервиса?

Подсказка: Подумайте о постепенном увеличении нагрузки до отказа.

**Ответ:**

- **Максимальную пропускную способность и устойчивость сервиса**
- Ошибки в синтаксисе Go
- Количество пользователей на UI
- Количество Docker-образов

#### 🔗 Шаг 07:

##### Вопрос 6

Сопоставьте нагрузочные сценарии с их описанием.

Подсказка: Один сценарий — постепенное увеличение нагрузки, другой — равномерная постоянная нагрузка.

**Ответ:**

- **Load Test** — Равномерная нагрузка на сервис для проверки стабильности

- **Stress Test** — Постепенное увеличение нагрузки до отказа
- **Spike Test** — Внезапный резкий рост числа запросов
- **Endurance Test** — Длительное тестирование для выявления утечек памяти

## 🔗 Шаг 08:

### Вопрос 7

Какие узкие места чаще всего выявляются при нагрузочном тестировании Go-сервисов?

Подсказка: Подумайте о ресурсах сервера и производительности базы данных.

**Ответ:**

- **CPU-пиковые нагрузки**
- **Медленные запросы к базе данных**
- Ошибки в комментариях к коду
- **Высокое использование памяти**

## 🔗 Шаг 09:

### Вопрос 8

Расположите шаги анализа метрик после нагрузки на Go-сервис:

Подсказка: Сначала собираем данные, потом анализируем и принимаем решения по оптимизации.

**Ответ:**

- Сбор метрик CPU, RAM, диска и сети
- Сбор метрик latency, throughput, error rate
- Анализ узких мест и причин замедлений
- Внедрение оптимизаций: кеширование, пул соединений, масштабирование

## 🔗 Шаг 10:

### Вопрос 9

Что поможет проверить отказоустойчивость сервиса при нагрузочном тестировании?

Подсказка: Попробуйте отключить один из экземпляров сервиса и наблюдать за поведением системы.

**Ответ:**

- **Остановка одного экземпляра и проверка работы оставшихся узлов**
- Проверка документации API
- Изменение Dockerfile
- Перезапуск сервиса без нагрузки

## 🔗 Шаг 11:

### Вопрос 10

Сопоставьте действия с целью оптимизации highload-сервиса после нагрузочного тестирования.

Подсказка: Кеширование ускоряет доступ к данным, а горизонтальное масштабирование добавляет мощности.

**Ответ:**

- **Кеширование (Redis/Memcached)** — Снижение нагрузки на базу данных
- **Пул соединений с БД** — Уменьшение затрат на установку соединений

- **Горизонтальное масштабирование** — Добавление новых экземпляров сервиса для увеличения пропускной способности
- **Асинхронная обработка задач** — Снижение задержек при обработке запросов